

E Rescue



Developed by

Areeba Fatima	UOC-IT-F2021-029
Fatima Ishtiaq	UOC-IT-F2021-044
Sami Ullah Khan	UOC-IT-F2021-041

Supervised By

Dr. Rashid Amin

Department of Computer Science & Information Technology

Faculty of Computing & Information Technology

The University of Chakwal

Bachelor of Science in Information Technology (2021-2025)

DECLARATION

We hereby declare that this project **E Rescue**, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software documentation and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Areeba Fatima
UOC-IT-F2021-029

Fatima Ishtiaq
UOC-IT-F2021-044

Sami Ullah Khan
UOC-IT-F2021-041

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS IT “**E Rescue**” was developed by “**Areeba Fatima, UOC-IT-F2021-029**”, “**Fatima Ishtiaq, UOC-IT-F2021-044**”, “**Sami Ullah Khan, UOC-IT-F2021-041**” under the supervision of “**Dr. Rashid Amin**” and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Information Technology.

Dr. Rashid Amin

Supervisor

Engr. Safia Bibi

External Examiner

Ms. Hina Iftikhar

HOD

Abstract

As the need for personal safety solutions grows, the e-rescue app is an important tool for improving emergency response. This app helps users quickly and reliably get help during critical situations. Traditional ways of seeking assistance can be slow and may lead to dangerous delays. The e-rescue app tackles these issues by offering features like real-time location sharing, an SOS alert button, and live video streaming. These tools are designed to speed up response times and enhance user safety. With an easy-to-use interface, individuals can navigate the app effortlessly, even in stressful moments, making it simple to call for help when needed. By using IoT technology, the app allows smooth communication between users and emergency responders, ensuring faster assistance. Additionally, machine learning helps the app learn from user behavior to improve its performance over time, making it more reliable.

Overall, the e-rescue app is a vital resource for personal safety and emergency preparedness. By empowering users to take charge of their safety, the app aims to make a real difference in how people respond to emergencies, ultimately saving lives and improving outcomes in critical situations.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Dr. Rashid Amin**” for personal supervision, advice, valuable guidance and completion of this project. We are deeply indebted to him for encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Member 01

Member 02

Member 03

Table of Contents

Chapter 1: Introduction.....	10
1.1. Project Background	10
1.2. Methodology and Software Lifecycle for this Project.....	2
1.3. Rationale behind Selected Methodology	2
1.4. Project objectives.....	2
1.5. Project Scope	3
1.6. Tools and Technologies	3
1.7. Project Summary	5
Chapter 2: Literature Review and Problem Definition.....	6
2.1. Literature Review	6
2.2. Analysis from Literature Review.....	7
2.3. Problem Statement.....	8
2.4. Product Functions	8
2.5. Chapter Summary	9
Chapter 3: Requirement Analysis	10
3.1. Functional Requirements	10
3.2. Non-Functional Requirements.....	12
3.3. Use Cases model.....	14
3.4 Chapter Summary:	18
Chapter 4: Design and Architecture.....	19
4.1. Proposed Architecture	19
4.2 Deliverables	19
4.3. Development Requirements:	20
4.4. Assumptions and Dependencies	20
4.5. UML Diagrams	22
Chapter 5: Project Implementation	44
5.1. Programming language.....	44
5.2. Framework	44
5.3. System Requirement.....	44
5.4. Chapter Summary	45
Chapter 6: Software Testing & Maintenance	46
6.1 Deriving Test	46
6.2 Test Environment.....	46

6.3	Testing identification	46
6.4	Test Procedure	47
6.5	Testing technique.....	48
6.6	Test Cases	49
6.7	Chapter Summary	52
Chapter 7: Conclusion and Future Work		53
7.1.	Discussion.....	53
7.2.	Conclusion	53
7.3.	Limitations	53
7.4.	Future work.....	54
App Screens.....		49
References.....		69

List of Figures

Fig 3.1 Use Case Diagram	13
Fig 4.1 Sequence Diagram.....	21
Fig 4.2 Block Diagram	24
Fig 4.3 Data Flow Diagram	27
Fig 4.3.1 DFD Level-0 Diagram.....	27
Fig 4.3.2 DFD Level-1 Diagram	27
Fig 4.4 Class Diagram	33
Fig 4.5 Activity Diagram.....	37

List of Tables

Table 3 Use Case Description Table.....	13
Table 3.1 Login.....	13
Table 3.2 Activate emergency sos button.....	13
Table 3.3 share real time location.....	13
Table 3.4 record audio	13
Table 3.5 receive weather alert.....	13
Table 3.6 Geofencing alerts	14
Table 3.7 Access offline mode	14
Table 3.8 Access safety tips and guides	14
Table 3.9 Access responder pannel.....	14
Table 3.10 Generate incident report	14
Table 3.11 Fall detection sensor	15
Table 3.12 Audio sensor recording.....	15
Table 3.13 Mechanic request.....	15

Chapter 1: Introduction

In our fast-paced world, personal safety is more important than ever. The e-rescue app helps people and families feel secure during emergencies. It has an Emergency SOS button to quickly call for help, real time location sharing to let loved ones know where you are, and live video streaming to share what's happening. The app also sends weather alerts to warn users about bad weather and works with wearable devices for easy tracking. With these features, the e-rescue app makes it easier for users to get help when they need it most.

1.1. Project Background

The e-rescue app was developed to address the increasing need for personal safety solutions in our fast-paced and often unpredictable world. As concerns about crime, natural disasters, and health emergencies grow, individuals and families are looking for reliable ways to protect themselves and ensure quick access to help when they need it. Traditional methods of seeking assistance, such as calling for help or relying on physical safety measures, may not always be effective in urgent situations. The e-rescue app aims to fill this gap by using technology to provide users with immediate support and peace of mind, making it easier for them to respond to emergencies.

The idea for the project came from the widespread use of smartphones, which have become essential parts of daily life. By integrating safety features into this familiar tool, the e-rescue app offers an easy-to-use solution that can be accessed quickly in times of crisis. We conducted thorough research to understand what users need most, focusing on features that enhance communication, location tracking, and emergency response. This way, the e-rescue app is designed to empower individuals to take control of their safety and stay connected with their loved ones, making it a vital tool for modern living.

1.2. Methodology and Software Lifecycle for this Project

To build the e-rescue application effectively, we used the Agile development method. Agile is known for being flexible, allowing for continuous improvement, and encouraging teamwork with users and stakeholders. Instead of creating the whole app at once, we broke the development into smaller parts called iterations or sprints. Each sprint results in a working feature of the app, which helps us get feedback regularly and make quick changes based on what users need and the goals of the project.

1.3. Rationale behind Selected Methodology

We chose the Agile method because it is flexible and allows us to improve the e-rescue app throughout the development process. Agile encourages regular communication with users and team members, helping us better understand their needs. By breaking the project into small steps, we can quickly deliver parts of the app and gather feedback. This approach allows us to fix problems and make necessary changes quickly, which is essential for an emergency rescue app where reliability and quick updates are very important.

1.4. Project objectives

The primary goals of the Rescue Assistance Application are as follows:

- **Provide Fast Emergency Alerts:** Create a quick and easy way for users to send SOS alerts to their emergency contacts with a single tap.
- **Enable Real-Time Location Sharing:** Allow users to share their location immediately, making it easier for emergency contacts or responders to locate them.
- **Integrate Community Support:** Notify nearby app users about emergencies, increasing the chances that someone can help in a timely manner.
- **Enhance User Safety and Confidence:** With features like live video streaming and AI-powered threat detection, the app will empower users to handle emergencies with more confidence.
- **Offer User-Friendly and Accessible Features:** Ensure the app is simple to use, compatible across devices, and responsive, so it works smoothly when users need it the most.

1.5. Project Scope

The goal of this project is to create an e-rescue mobile application that improves personal safety and emergency response. The app will allow users to quickly send SOS alerts and share their real-time location with trusted contacts and emergency services. Key features will include emergency notifications, live location tracking, and access to nearby help resources. The app will be available on both Android and iOS devices and will function offline to ensure usability without internet access. The focus will be on developing a user-friendly interface that makes it easy to access safety features quickly. This project will cover the design, development, and testing of the app to ensure it effectively supports users in emergencies

1.6. Tools and Technologies

The development of the E-Rescue App required a combination of software platforms and hardware components to ensure real-time emergency response, location tracking, and alert delivery. Below is a detailed overview of the tools and technologies used:

Software Tools

1. Flutter

Used to build the cross-platform mobile application with a single codebase. It provides a fast and responsive user interface.

2. Dart

The programming language used with Flutter for frontend development. It supports real-time interactions and asynchronous operations.

3. Android Studio

Served as the primary IDE for developing, debugging, and testing the mobile app, along with virtual device testing.

4. Firebase

Used for real-time database, user authentication, cloud storage, and push notifications. It enables secure and reliable backend operations.

5. Arduino IDE

Used to program the ESP8266 and other microcontroller-based components. It helped integrate hardware functions such as sensor input and GSM-based communication.

6. Google Maps API

Integrated to display user location and implement geofencing functionalities.

Hardware Components

1. ESP8266 Wi-Fi Module

Used to fetch the current GPS location and send it via internet or SMS. It plays a key role in linking the hardware system with the mobile app or cloud.

2. GSM SIM800L Module

Used to send SMS alerts and make emergency calls without requiring an internet connection. It ensures offline communication in case of no network.

3. Accelerometer Sensor (MPU6050)

Used for fall detection. It senses sudden movements or impacts and triggers an automatic SOS alert when a fall is detected.

4. Microphone

Used to record emergency audio, which can be stored or sent to responders for better situation awareness.

1.7. Project Summary

The e-rescue app is designed to improve personal safety and emergency response. It provides users with a reliable way to connect with their families and emergency services during critical situations. The app focuses on quick access to help and effective communication, ensuring that users receive timely assistance when they need it most. By combining various features, the e-rescue app aims to create a comprehensive safety solution that helps users feel secure and supported in emergencies.

Chapter 2: Literature Review and Problem Definition

In today's world, personal safety during emergencies is a significant concern. Many existing rescue systems struggle with slow response times and poor communication. This project aims to create an e-rescue app that uses technology to improve emergency response. The app will provide quick access to help, share real-time locations, and enhance communication between users and emergency services. By addressing these issues, the e-rescue app seeks to make rescue operations faster and more effective.

2.1. Literature Review

Feature	Life 360	I First Aid	Rescue 1122	Red Panic Button	B Safe
Basic Features	Location tracking, driving reports, SOS alerts	First aid guides, medical info, emergency contacts	Emergency services contact, location tracking	One-touch emergency alerts, location tracking	SOS alerts, location sharing, check-in feature
Premium Features	Location history, Crash Detection, Driver Safety	Offline first aid guides, Premium medical features	Access to Rescue 1122 services with a dedicated SOS button	Custom panic alerts, Multiple contacts, Voice message	Advanced check-ins, Multiple alerts, Location history
Geofencing	Yes, for location-based notifications	No	No	No	Yes
Safe Zone/Alerts	Yes, set boundaries to track family members	No	No	Yes, sends location and alerts to contacts	Yes, sends alerts to contacts and authorities

Offline Function	Limited (Offline map access in premium version)	Yes, offline first aid guides	Limited, needs an internet connection for full functionality	Limited without internet connection	Yes, SOS feature works even without Internet
Voice Calling	Yes, in premium version (for emergencies)	No	No	Yes	Yes, in premium version
Emergency Contact Sharing	Yes, with location sharing	Yes, with location sharing	No	Yes, sends alerts to predefined contacts	Yes, sends alerts to predefined contacts
Subscription Price	Free, Premium starts at \$7.99/month	Free, Paid version available for more features	Free, no premium version available	Free	Free, Premium starts at \$3.99/month
Available on Platforms	iOS, Android [1]	iOS, Android [2]	iOS, Android [3]	iOS, Android [4]	iOS, Android [5]

2.2. Analysis from Literature Review

The review of rescue and safety apps shows that they offer many helpful features, such as tracking locations, sending emergency alerts, and providing medical information. However, many of these apps have limitations, especially when it comes to working offline, which is important during emergencies when internet access might not be available. Additionally, features like geofencing and sharing emergency contacts vary widely in their effectiveness. This analysis highlights a clear opportunity for the e-rescue app to combine the best features from existing apps while improving their weaknesses, particularly by enhancing offline functionality and offering better medical assistance resources. This way, the e-rescue app can become a more effective and user-friendly tool for handling emergencies.

2.3. Problem Statement

In today's world, personal safety and emergency response have become critical concerns due to the increasing frequency of accidents, natural disasters, and unforeseen crises. Although several rescue apps like Life360, I First Aid, Rescue 1122, Red Panic Button, and B Safe exist, they lack comprehensive integration of features necessary for efficient and reliable emergency handling.

Existing applications face limitations such as high battery consumption, lack of offline functionality, absence of advanced safety measures like geofencing with danger zone alerts, and limited communication prioritization during emergencies. Moreover, features like automatic location updates in unsafe zones, live video streaming for better situational understanding, and optimized battery usage during crises are absent or not effectively implemented.

This fragmented approach leaves users juggling between multiple apps to fulfill their safety and emergency needs, reducing overall efficiency during critical moments. Therefore, there is a pressing need for a unified rescue solution that combines the strengths of existing apps while addressing their limitations to provide a seamless and reliable emergency response platform. [6]

2.4. Product Functions

The key functions of the proposed system include:

- Family Circle for secure communication among family members
- One-Touch SOS Button for immediate alerts
- Live Location Sharing with designated contacts
- Audio Recording feature for emergencies
- Notifications for Severe Weather Alerts
- Geofencing to automatically notify when entering danger zones
- Offline Mode to access essential features without internet
- Battery Saver Mode to prolong app functionality
- Access to Personal Safety Tips and Emergency Guides
- Priority Calling for emergency contacts

- Responder Dashboard for emergency personnel
- Automatic Incident Report Generation
- Fall Detection Sensor to identify falls
- Audio Sensor to capture sounds related to incidents

2.5. Chapter Summary

In this chapter, we review existing research on emergency response systems and personal safety apps. The literature review covers different methods and technologies designed to improve how emergencies are managed and responded to. The analysis highlights important trends and gaps in current solutions, showing the need for better communication, real-time data sharing, and easy-to-use designs in emergency apps. We then present the problem statement, which outlines the challenges people face during emergencies, such as slow response times, lack of awareness, and poor communication with emergency services.

Finally, we describe the main features of the proposed e-rescue app, which aims to solve these problems by including tools like a Family Circle for secure communication, an Emergency SOS Button, real-time location sharing, and other safety features to help users and improve emergency responses.

Chapter 3: Requirement Analysis

3.1. Functional Requirements

Functional Requirements of our E rescue app are following:

1. Family Circle

- **Detail:** Users can create a secure network of family members within the app, allowing them to connect and communicate easily.
- **Purpose:** This feature enhances communication and safety during emergencies, ensuring that family members can quickly reach out to one another.

2. Emergency SOS Button

- **Detail:** A prominently displayed one-touch button that instantly alerts emergency services and designated contacts when activated.
- **Purpose:** This feature provides immediate assistance in case of a crisis, ensuring that help is summoned without delay.

3. Real-Time Location Sharing

- **Detail:** Users have the ability to share their live location with family members or emergency responders in real time.
- **Purpose:** This ensures that help can locate the user quickly when needed, improving response times during emergencies.

4. Audio Recording

- **Detail:** Users can capture audio recordings during emergencies to document events as they unfold.
- **Purpose:** This feature serves to provide evidence to authorities and helps in creating a clear record of the incident.

5. Weather Alerts

- Detail: Users receive timely notifications about severe weather conditions occurring in their area, such as storms or floods.
- Purpose: This helps users stay informed and safe by allowing them to take necessary precautions in response to changing weather conditions.

6. Geofencing

- Detail: The app automatically updates the user's location if they enter a predefined danger zone, such as an area known for high crime or natural disasters.
- Purpose: This feature alerts family members and emergency contacts about the user's whereabouts, enhancing safety and awareness.

7. Offline Mode

- Detail: Users can access essential features and information even when there is no internet connection available.
- Purpose: This ensures that the app remains functional during network outages, allowing users to access critical tools in emergencies.

8. Personal Safety Tips and Guides

- Detail: Users can access a comprehensive library of safety tips and emergency response guides tailored to various situations.
- Purpose: This provides users with valuable information and strategies to handle emergencies effectively and safely.

9. Responder Panel

- Detail: A dedicated interface for emergency responders that allows them to access critical information about the situation and the user.

- Purpose: This facilitates a more effective and informed response to emergencies, improving overall outcomes.

10. Incident Report Generation

- Detail: The app generates detailed reports documenting incidents, including user actions and responses.
- Purpose: This provides valuable documentation for review and analysis, helping to improve future emergency responses.

11. Fall Detection Sensor

- Detail: The app includes a sensor designed to identify falls and automatically alert designated contacts.
- Purpose: This feature ensures that emergency contacts are notified in case of a fall, allowing for timely assistance.

12. Audio Sensor

- Detail: The app is equipped with an audio sensor that records sounds associated with events during emergencies.
- Purpose: This provides context and evidence related to incidents, helping authorities understand the situation better

3.2. Non-Functional Requirements

As we develop the e-rescue app, it is essential to consider non-functional requirements that will significantly impact user experience and system performance. These requirements address how the app behaves under various conditions, ensuring it is efficient, reliable, and user-friendly. By focusing on these aspects, we can create a solution that not only meets functional needs but also provides a seamless and dependable experience for users in emergency situations. Below are the key non-functional requirements that will guide our development process.

3.2.1 Usability: The app is designed with an intuitive interface that prioritizes ease of use, even in high-pressure situations. This means that users can quickly navigate through the app to access critical features like the SOS button or location sharing. The layout is simple and straightforward, minimizing the need for extensive instructions or training. This focus on usability ensures that users can act swiftly and confidently during emergencies, which is essential for their safety.

3.2.2 Performance: The app is engineered for high performance, ensuring that SOS alerts and location sharing features respond in under 2 seconds. This rapid response time is crucial in emergency situations, where every second counts. Users can expect immediate feedback when they activate the SOS button or share their location, which enhances the overall effectiveness of the app in critical moments.

3.2.3 Reliability: With a target uptime of 99.9%, the app is built to be highly reliable, ensuring that it is available whenever users need it. It can handle multiple users simultaneously without any noticeable performance issues, which is important during emergencies when many people may be trying to access the app at the same time. This reliability fosters user trust and confidence in the app's ability to function effectively under pressure.

3.2.4 Scalability: The app is designed to scale efficiently, meaning it can accommodate an increasing number of users without experiencing performance drops. As more individuals join the platform, the app maintains its responsiveness and speed, ensuring that all users can access its features without delays.

3.2.5 Compatibility: The app is compatible with both Android and iOS operating systems, allowing it to function seamlessly across a variety of devices, including smartphones and tablets. This broad compatibility ensures that users can access the app regardless of their device preference, making it more accessible to a wider audience and enhancing its effectiveness in emergency situations.

3.2.6 Low Power Consumption: The app is optimized for low power consumption, which is critical during emergencies when users may not have immediate access to charging options.

3.3. Use Cases model

3.3.1 Use Case Diagram

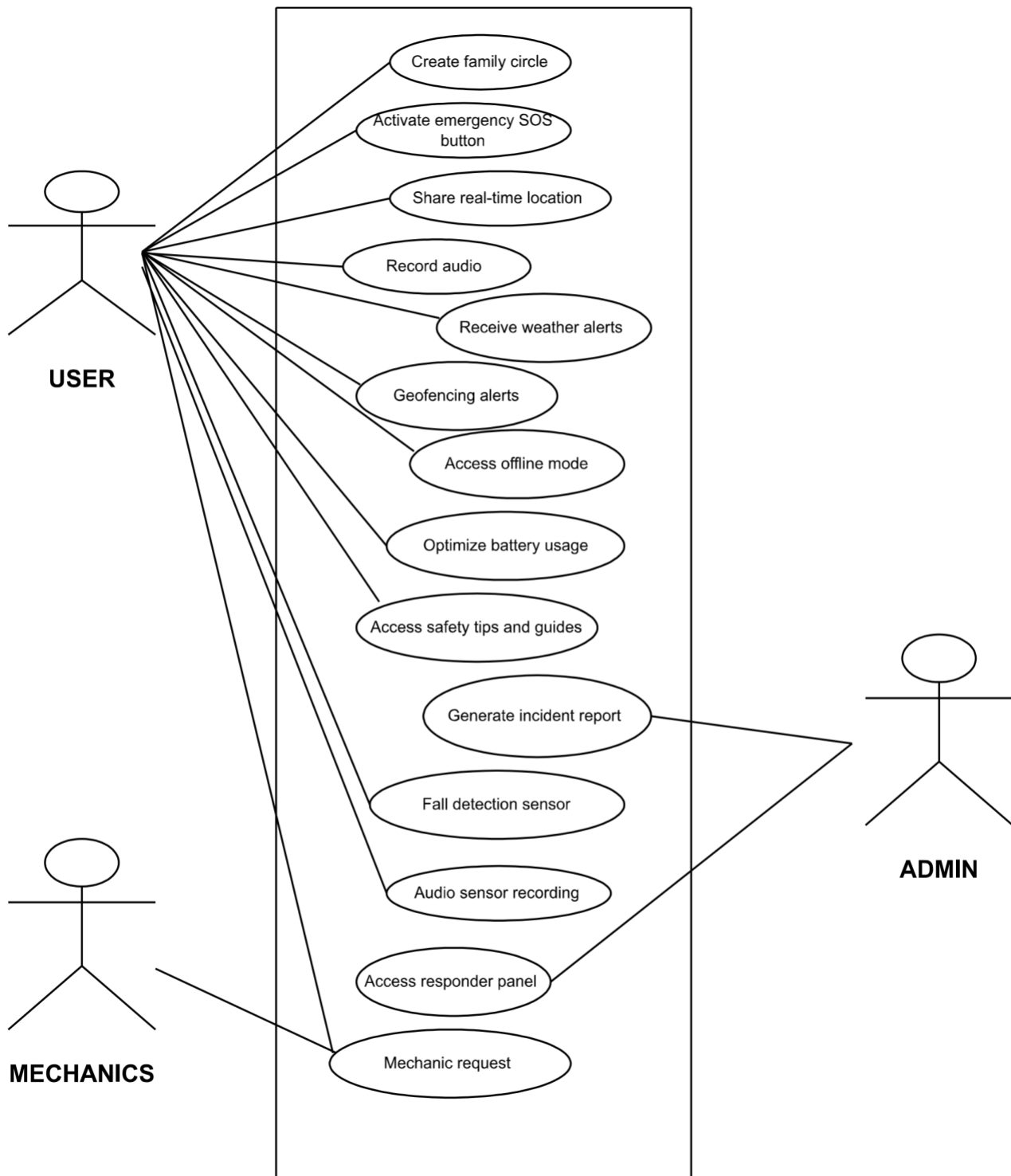


Figure 3.1: Use Case Diagram

3.3.2 Use Case Description

Table 3.1: Login

Use Case ID	3.1
Use Case Name	Create Family Circle
Actors	User
Description	The user creates a secure group of family members within the app.
Preconditions	User must be logged in.
Postconditions	Family circle is created and saved in the system.

Table 3.2: Activate Emergency SOS Button

Use Case ID	3.2
Use Case Name	Activate Emergency SOS Button
Actors	User
Description	Sends immediate SOS alert to emergency contacts and services.
Preconditions	Emergency contacts must be set.
Postconditions	SOS alerts are sent to all concerned parties.

Table 3.3: Share Real-Time Location

Use Case ID	3.3
Use Case Name	Share Real-Time Location
Actors	User
Description	Shares live user location with family or responders.
Preconditions	Location services must be enabled.
Postconditions	Location shared successfully.

Table 3.4 : Record Audio

Use Case ID	3.4
--------------------	------------

Use Case Name	Record Audio
Actors	User
Description	Captures emergency audio evidence.
Preconditions	Microphone access granted.
Postconditions	Audio recorded and saved.

Table 3.5 : Receive Weather Alerts

Use Case ID	3.5
UseCase Name	Receive Weather Alerts
Actors	User
Description	Notifies user about dangerous weather.
Preconditions	Location and alert settings enabled.
Postconditions	User receives weather updates.

Table 3.6 : Geofencing Alerts

Use Case ID	3.6
Usecase Name	Geofencing Alerts
Actors	User
Description	Triggers alerts in predefined danger zones.
Preconditions	Geofencing set up and GPS on.
Postconditions	Alert sent when boundary is crossed.

Table 3.7: Access Offline Mode

Use Case ID	3.7
Use Case Name	Access Offline Mode
Actors	User
Description	Uses basic features without internet.
Preconditions	Offline content cached.
Postconditions	Features work without internet

Table 3.8: Access Safety Tips and Guides

Use Case ID	3.8
Use Case Name	Access Safety Tips and Guides
Actors	User
Description	User reads emergency tips and safety guides.
Preconditions	App content updated.
Postconditions	Tips are displayed to user.

Table 3.9 :Access Responder Panel

Use Case ID	3.9
Use Case Name	Access Responder Panel
Actors	Admin
Description	Admin views user emergency info.
Preconditions	Admin logged in.
Postconditions	Incident info is accessed.

Table 3.10 : Generate Incident Report

Use Case ID	3.10
Use Case Name	Generate Incident Report
Actors	User, Admin
Description	Creates an incident report.
Preconditions	Incident occurred.
Postconditions	Report saved or exported.

Table 3.11 : Fall Detection Sensor

Use Case ID	3.11
Use Case Name	Fall Detection Sensor
Actors	User
Description	Detects falls and alerts emergency contacts.

Preconditions	Sensor enabled.
Postconditions	Alert sent after fall.

Table 3.12 : Audio Sensor Recording

Use Case ID	3.12
Use Case Name	Audio Sensor Recording
Actors	User
Description	Records ambient sounds during emergencies.
Preconditions	Microphone access granted.
Postconditions	Audio clip saved.

Table 3.13 : Mechanic Request

Use Case ID	3.13
Use Case Name	Mechanic Request
Actors	User, Mechanic
Description	Sends request for vehicle/mechanical help.
Preconditions	Location enabled and user logged in.
Postconditions	Request sent to mechanic

3.4 Chapter Summary:

This chapter presented a comprehensive analysis of the requirements for the E-Rescue mobile application. It outlined both the functional and non-functional requirements essential for ensuring the system's reliability, usability, and performance.

The functional requirements focused on the core features of the application such as emergency alert generation, location sharing, user authentication, and real-time communication with rescue services. The non-functional requirements covered critical aspects such as performance, security, scalability, and especially availability, ensuring the system remains accessible and dependable during emergencies.

Chapter 4: Design and Architecture

4.1. Proposed Architecture

The proposed architecture of the E-Rescue application is based on a **Client-Server model**, ensuring real-time communication, scalability, and security during emergency situations.

Client Side (Mobile App)

- Acts as the user interface, allowing individuals to:
- Trigger SOS alerts
- Stream audio/video
- Share real-time location
- Receive weather alerts and safety tips
- Works even in offline mode by storing data locally until a connection is restored

Server Side (Backend & Database)

- Receives and processes SOS data from clients
- Stores user information, incident logs, and media files
- Sends notifications to registered emergency contacts and responders
- Provides access to a web-based Responder Panel for monitoring incidents
- Generates and stores incident reports for review

Communication

- The client communicates with the server using secure protocols (e.g., HTTPS, REST APIs)
- The server pushes notifications to clients via Firebase or similar services

4.2 Deliverables

The project aims to deliver the following key outcomes:

1. **Emergency SOS Module:** Implementation of a one-touch SOS alert system that instantly notifies emergency contacts and responders with the user's location and status.

2. **Real-Time Location Tracking:** Development of a GPS-based live location sharing system to assist responders and family members in locating the user quickly during emergencies.
3. **Fall Detection and Audio Sensors:** Deployment of motion and audio sensor-based features to detect accidents (like falls) and trigger automated alerts.
4. **Responder Dashboard:** Creation of a dedicated panel for emergency response teams to monitor alerts, track user locations, and access live feeds.
5. **Offline Emergency Mode:** Development of a module enabling access to key safety features without internet connectivity, ensuring reliability in all conditions.
6. **8. Personal Safety Guide:** Inclusion of a built-in library of emergency tips and response guides to educate users on safety practices.

4.3. Development Requirements:

Programming Languages

- **Dart (with Flutter):** For cross-platform mobile app development (Android & iOS)

Libraries

- **Flutter SDK:** UI toolkit for building the mobile interface
- **Firebase:** For real-time database, push notifications, and cloud messaging
- **Google Maps API:** For real-time location tracking and geofencing

APIs & External Tools

- **Device Sensors (Accelerometer, Gyroscope):** For fall detection
- **Weather API (like OpenWeatherMap):** To display weather alerts

4.4. Assumptions and Dependencies

The assumptions for the project include:

- Users (general public and emergency responders) have access to Android smartphones with basic hardware capabilities (GPS, camera, microphone, accelerometer, etc.).

- Users have a stable internet connection to access features like real-time location sharing, live video streaming, and push notifications.
- Users understand basic mobile operations, such as enabling location access, camera permissions, and navigating app interfaces.
- The app will initially support only Android devices (Android 10.0 and above).
- Users will provide accurate personal and emergency contact information during registration.
- Users will use the app ethically and responsibly, only triggering SOS alerts in genuine emergencies.
- Emergency responders accessing the Responder Panel are registered personnel with authorized access.

Dependencies:

Firebase services are required for managing user data, incident logs, push notifications, and media storage.

A reliable internet connection is essential for real-time SOS alerts, live location updates, and audio/video transmission.

The app relies on the Flutter SDK and its third-party libraries for front-end and service logic.

The proper functioning of device-level components such as GPS, accelerometer, microphone, is critical for SOS, fall detection.

Cloud APIs (e.g., Google Maps API, Weather API) are needed for geolocation, geofencing, and weather alert features.

User engagement and quick response from emergency contacts or responders are necessary for successful emergency handling.

The project depends on the stability and availability of third-party services like Firebase, Google Maps, and external APIs throughout development and post-deployment

4.5. UML Diagrams

UML is known as unified model language and used to make static structured diagrams which will show how system will work. There are many UML diagrams but some of them are:

4.5.1. Sequence Diagram

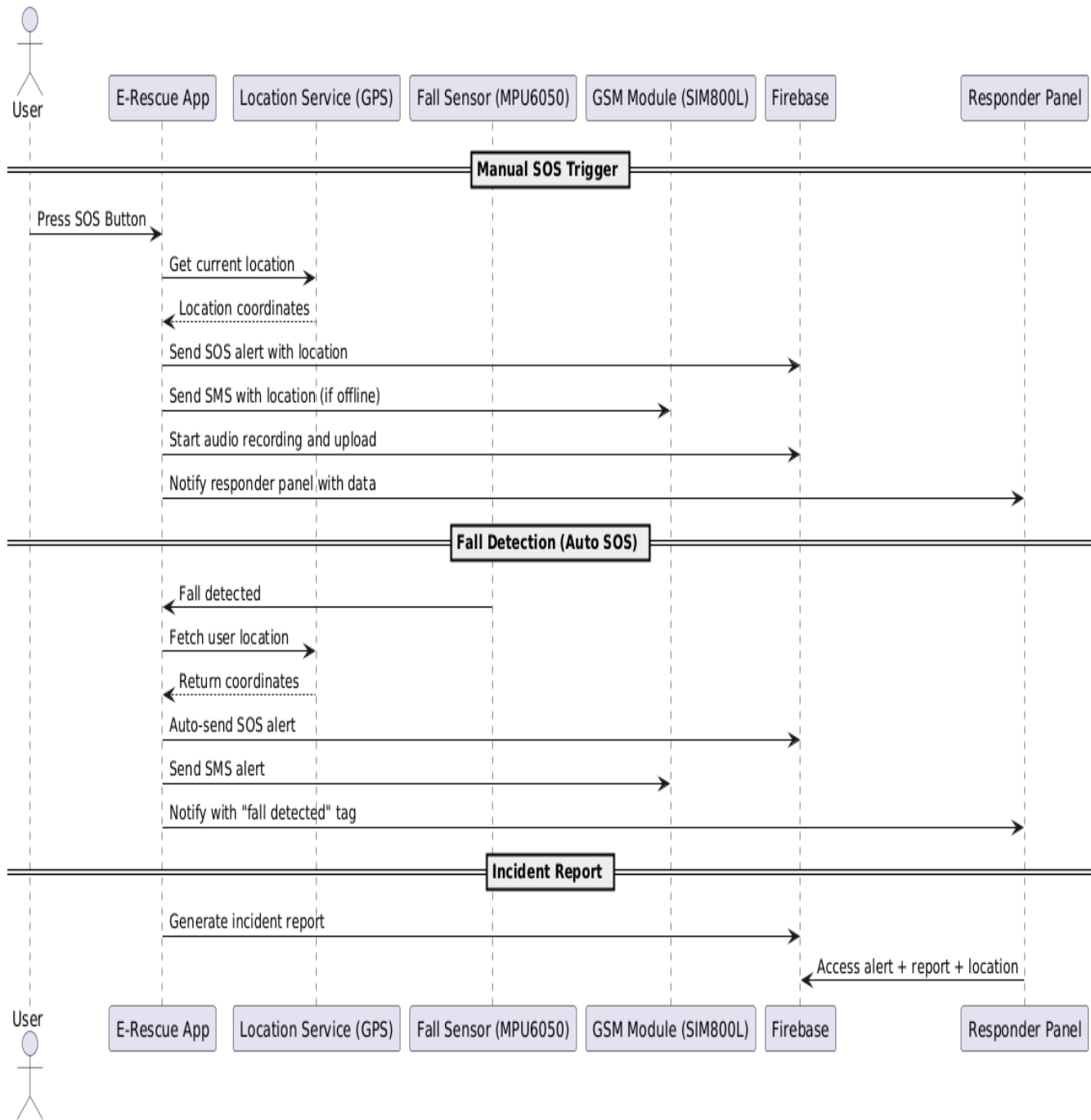


Figure 4.1: Sequence Diagram

Description of Sequence Diagram

The sequence diagram illustrates how the components of the E-Rescue App interact during emergency situations. The system supports both user-initiated alerts and automatic fall-triggered alerts, and it is designed to function under both online and offline conditions using Firebase and GSM-based SMS respectively.

Main Actors and System Components:

- **User:** The end user who might be facing an emergency. This actor initiates manual SOS alerts and is the subject of automatic fall detection.
- **E-Rescue App:** The central system that controls the app's operations, user interface, and communication between components.
- **Location Service (GPS):** Used to fetch the user's live location when an SOS is triggered.
- **Fall Sensor (MPU6050):** A hardware sensor that detects a sudden fall or physical impact and initiates the automatic emergency response.
- **GSM Module (SIM800L):** A hardware component that sends emergency SMS alerts when internet access is unavailable.
- **Firebase (Cloud):** A cloud-based backend platform used for real-time alert delivery, data storage, audio uploads, and report generation.
- **Responder Panel:** A web or mobile interface accessed by emergency contacts or officials to monitor incoming alerts and take action.

Manual SOS Alert Sequence (User-Initiated Flow):

1. User presses the SOS button inside the E-Rescue App.
2. The app first interacts with the GPS module to retrieve the user's current coordinates.
3. Once the location is received, the E-Rescue App sends an SOS alert to Firebase, including the user ID, time, and location.
4. If there is no internet, the SIM800L GSM module is used to send the same alert via SMS to predefined emergency contacts.

5. At the same time, the app starts audio recording, which can provide context or evidence about the emergency.
6. The recorded audio is either:
 - Uploaded to Firebase if online, or
 - Stored locally on the device for later upload if offline.
7. The Responder Panel receives a real-time alert notification from Firebase, displaying:
 - The user's name and location
 - Timestamp of the alert
 - Audio (if available)
 - Status of the network (online/offline)

Automatic SOS Alert via Fall Detection:

1. The MPU6050 sensor continuously monitors the user's movement.
2. If a sudden fall or impact is detected, it sends a signal to the E-Rescue App.
3. The app immediately fetches the user's location via GPS and prepares the SOS alert.
4. The alert is sent:
 - To Firebase, if internet is available, or
 - Via GSM/SMS, if the device is offline.
 - The SOS message includes a flag such as "Fall Detected" so that the responders know this was not manually triggered and may involve unconsciousness or injury.
 - The Responder Panel displays this fall-tagged alert with all relevant information, ensuring quick action by the emergency contact or authority.

Incident Report Generation:

1. After any emergency event (manual or fall-based), the app generates a detailed incident report.
2. This report may include:

- User identity
- Location coordinates
- Date and time of alert
- Fall detection flag
- Audio recordings

3. The report is uploaded to Firebase and can be accessed later by both the user and the responder for reference or legal use.

4.5.2. Block Diagram

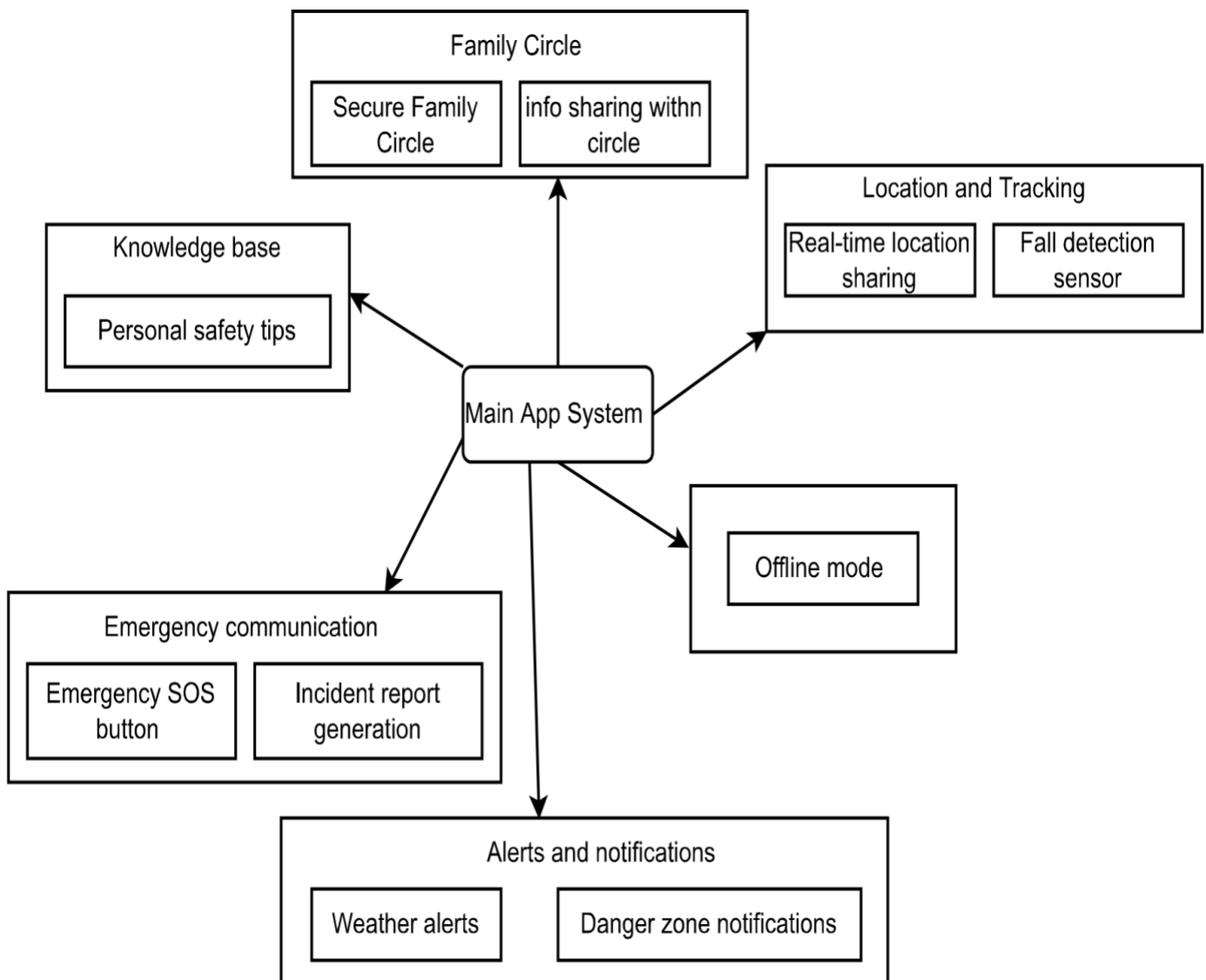


Figure 4.2: Block Diagram

Block Diagram Description:

Main App System (Central Controller)

At the core of the diagram is the **Main App System**, which acts as the **central hub** that manages and connects all other functional blocks. It handles:

- Data flow between components
- Sensor event handling (e.g., fall detection)
- Triggering of alerts and notifications
- Communication with external devices and services (e.g., Firebase, GSM modules)

This core is responsible for **synchronizing** all app features and ensuring that data such as location, alerts, and reports are processed in real time and sent to the relevant parties (family members, responders, or storage systems).

Family Circle Module

- **Secure Family Circle:** Allows users to add trusted contacts to a private circle. Only people within this circle can receive emergency alerts and location data.
- **Info Sharing Within Circle:** Facilitates the automatic sharing of the user's emergency status, GPS location, and alerts directly with circle members.
- **Importance:** This ensures quick help from family or friends before emergency responders arrive.

Location and Tracking Module

- **Real-time Location Sharing:** Continuously updates and sends the user's GPS coordinates to the app backend and trusted contacts.
- **Fall Detection Sensor:** Uses a motion sensor (e.g., MPU6050) to detect a fall automatically. If a fall is detected, the SOS alert is triggered automatically.
- **Functionality:** This module is critical for **tracking emergencies without user input**, especially in cases where the user is unconscious or immobile.

Emergency Communication Module

- **Emergency SOS Button:** A one-tap button that users press during an emergency to send alerts immediately.
- **Incident Report Generation:** After an SOS is triggered, the app collects information such as location, time, audio, and fall status to generate a detailed incident report. This report can later be accessed by admins or responders for evidence or review.
- **Goal:** Enable quick and efficient communication between the victim and the emergency network.

Offline Mode

- Ensures that the E-Rescue app can still function **without internet connectivity**. In such cases:
 - Alerts are sent using the **SIM800L GSM module**
 - Location and basic data are shared via **SMS**
- **Value:** This makes the app reliable even in remote areas or during disasters when networks are down.

Alerts and Notifications Module

- **Weather Alerts:** The app pulls weather updates from external sources and sends timely warnings to the user, such as flood alerts, storms, or heatwaves.
- **Danger Zone Notifications:** If the user enters a high-crime or disaster-prone area (geofenced), the app automatically notifies the user and their family circle.
- **Safety Benefit:** These alerts help users **avoid risk** and take **preventive action** even before an incident occurs.

Knowledge Base Module

- **Personal Safety Tips:** A collection of short, helpful tips that educate users on how to handle emergencies like fires, earthquakes, or attacks.

- **Emergency Response Guides:** In-depth step-by-step instructions for various scenarios, helping users make informed decisions until help arrives.

4.5.3. Data Flow Diagram

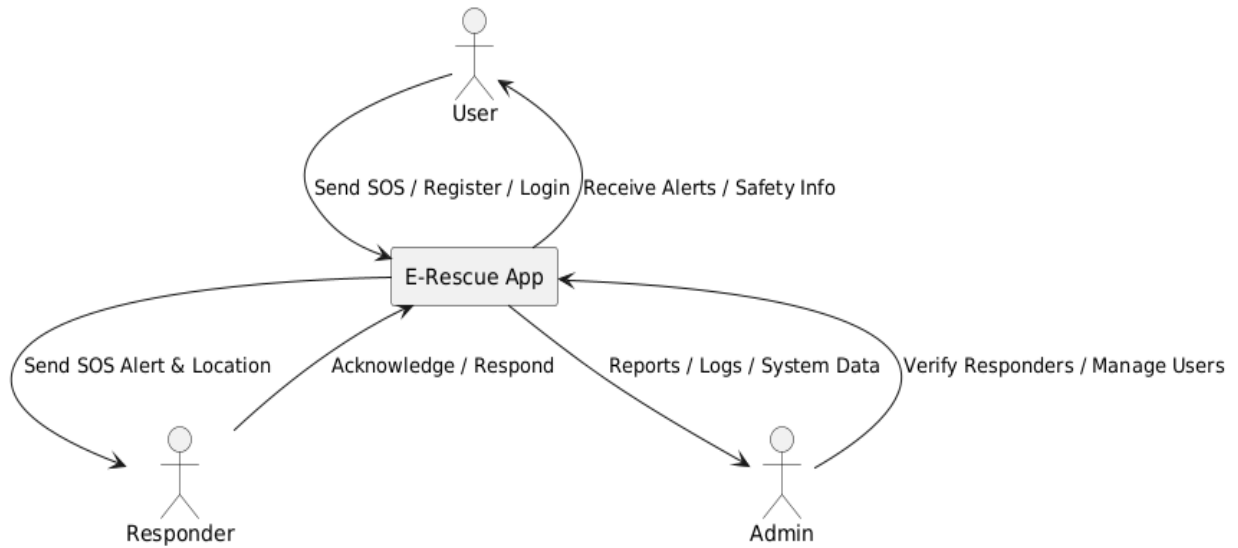


Figure 4.3: DFD Level-0

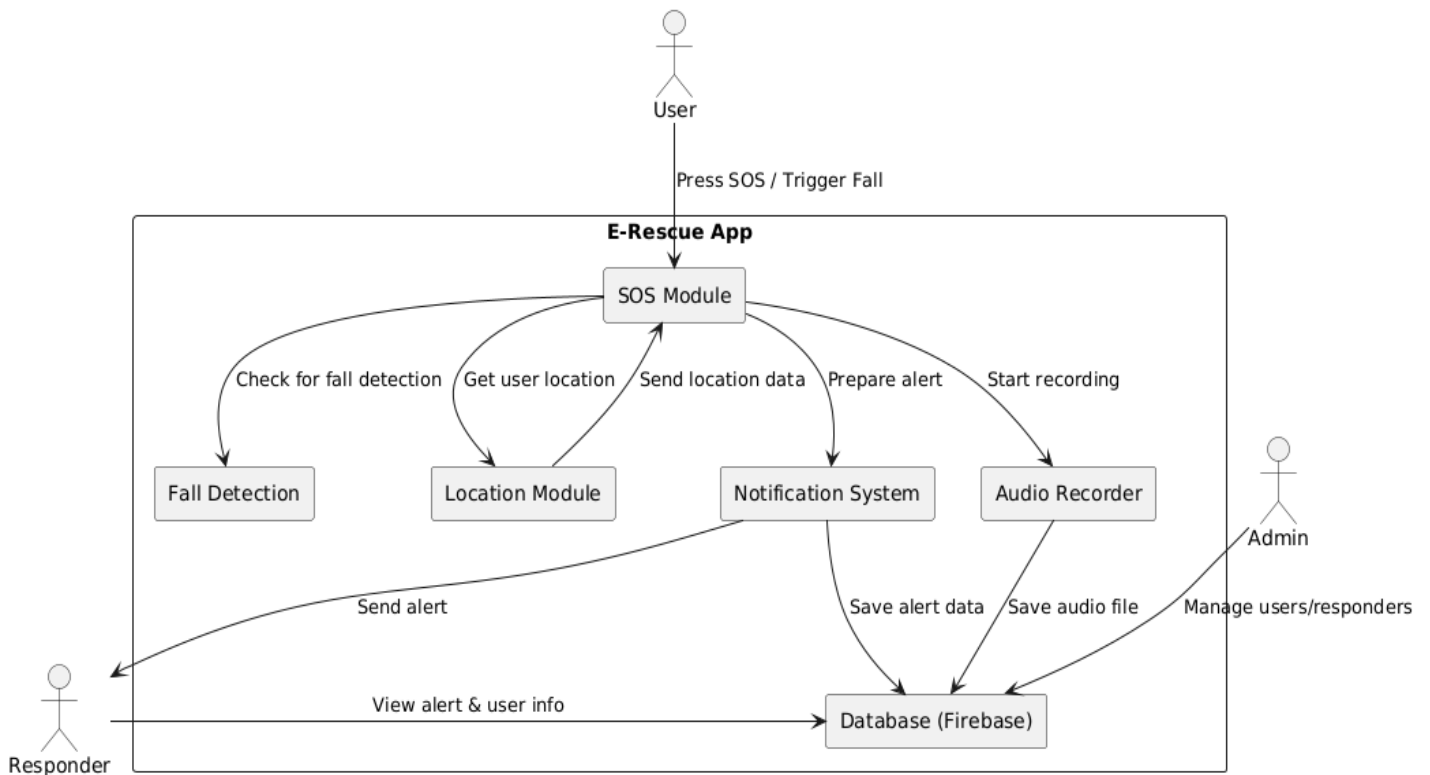


Figure 4.4: DFD Level-1

Description of DFD Level 0:

The Level 0 Data Flow Diagram is the top-level representation of the E-Rescue Application, presenting the system as a single unified process. It shows the external entities interacting with the system and the data flowing between them. This diagram does not describe internal modules but instead emphasizes the scope of the system and its data exchange with the outside world.

Central Process: "E-Rescue App"

The E-Rescue App is the **main process** in the Level 0 diagram. It is responsible for handling all the key functionalities such as:

- Receiving emergency requests (SOS),
- Sharing real-time location,
- Detecting falls via sensors,
- Sending alerts via cloud or SMS,
- Recording audio,
- Generating incident reports,
- Managing user and responder data.

It acts as the bridge between users in danger, emergency responders, and system administrators.

External Entities and Interactions

1. User

Description:

The primary end-user of the application who may be in an emergency situation.

Inputs to the System:

- Registers/logs into the app with credentials.
- Adds emergency contacts.
- Triggers SOS manually.

- Provides sensor inputs (like movement/fall).
- Shares location (automatically or manually).

Outputs from the System:

- Confirmation of alert delivery.
- Real-time safety status updates.
- Safety tips and weather alerts.
- Incident history or reports (for reference).
- Feedback notifications if alerts are received and responded to.

Role:

Users are at the center of the app. Their actions (like triggering SOS) begin most of the data flows in the system.

2. Responder

Description:

This entity refers to individuals or authorities (family members, police, medical responders) who receive and respond to emergency alerts.

Inputs from the System:

- Emergency alerts triggered by users.
- Real-time location of the user.
- Access to audio recordings (if shared).
- Incident reports detailing what happened.

Outputs to the System:

- Acknowledgment of received alerts.
- Responding with assistance updates.
- Status of rescue efforts or communication back to the user.

Role:

The responder ensures that once an emergency alert is sent, the right people take action. They interact with the app via a separate panel/dashboard.

3. Admin**Description:**

The Admin is the backend authority who manages, monitors, and maintains system stability and data accuracy.

Inputs to the System:

- Verifies responder profiles and authorizes access.
- Manages system settings and modules.
- Adds/removes users or responders if necessary.
- Uploads safety tips or updates.

Outputs from the System:

- Accesses logs, usage history, and incident reports.
- Receives alerts if system abuse or failure occurs.
- Monitors responder activity and user feedback.

Role:

The Admin ensures proper functioning of the app and maintains control over sensitive data, security, and permissions.

Description DFD Level 1:**1. SOS Module**

Function: Handles emergency alerts triggered by the user.

Data Input:

- Triggered manually by the user via the SOS button.
- Triggered automatically by the fall detection sensor.

Data Output:

- Sends SOS alert data to the Notification System.
- Initiates location and audio recording modules.
- Stores the alert in the database for future reports.

2. Fall Detection Module

Function: Monitors physical movement using the MPU6050 sensor.

Data Input:

- Continuously receives motion and acceleration data from the sensor.

Data Output:

- Triggers the SOS Module automatically when a fall is detected.
- Flags the incident with a “fall detected” label.

3. Location Module

Function: Fetches the user’s real-time location using the device's GPS or ESP8266 (if connected to hardware).

Data Input:

- Request from the SOS Module.

Data Output:

- Sends current coordinates to the Notification System and Database.
- Helps responders locate the user precisely.

4. Notification System

Function: Sends emergency alerts and updates.

Data Input:

- Receives alert and location data from the SOS Module.

Data Output:

- Sends push notification via Firebase (if online).
- Sends SMS via GSM SIM800L (if offline).
- Notifies the Responder Panel.
- Updates the user about the status of their alert.

5. Audio Recorder

Function: Records audio during emergency situations.

Data Input:

- Triggered by the SOS Module during an emergency.

Data Output:

- Saves audio locally or uploads it to Firebase (if connected).
- Links it to the specific alert in the database.

6. Incident Report Generator

Function: Compiles emergency data into a structured report.

Data Input:

- Alert time, location, fall status, audio, and user ID.

Data Output:

- Stores a report in the database.
- Makes it accessible to the Admin and Responder.

7. Database (Firebase)

Function: Stores and manages all system data.

Stores:

- User credentials, emergency contact lists.
- Alert history, location data, audio recordings.
- Fall detection logs, incident reports.

Used by:

- SOS Module, Notification System, Responder Panel, Admin Panel.

8. Responder Panel

Function: Interface used by responders to monitor and act on user alerts.

Data Input:

- Fetches alert and user data from the database.

Data Output:

- Displays emergency alerts, audio recordings, and incident reports.
- Allows responders to acknowledge and track incidents.

9. Admin Panel

Function: Manages the backend system.

Data Input:

- Allows the admin to view user activity, verify responders, and manage content.

Data Output:

- Shows logs, reports, and account data from the database.

4.5.4. Class Diagram

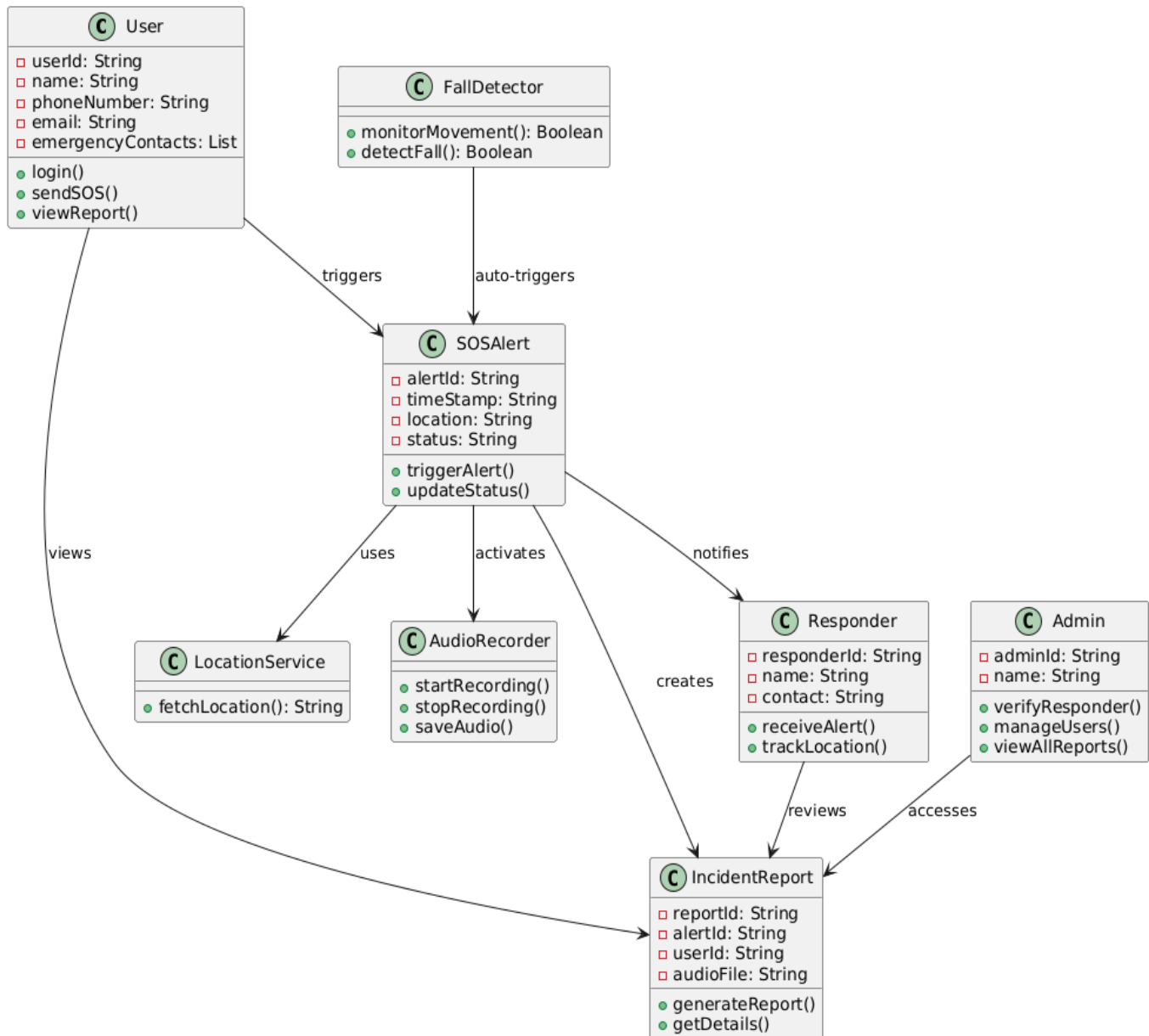


Figure 4.5: Class Diagram

Description of Class Diagram;

The **Class Diagram** of the **E-Rescue Application** represents the structure of the system in terms of its **classes**, their **attributes**, and **methods**, as well as the **relationships** between them. It gives a static view of how the objects in the system are organized and interact.

This diagram is essential for understanding how the application components are built and how they collaborate to provide emergency assistance through location sharing, fall detection, alert notifications, and report generation.

Main Classes and Their Responsibilities

1. User Class

Purpose: Represents the end-user of the app who may need emergency assistance.

Attributes:

- **userId:** A unique identifier for each user.
- **name, phoneNumber, email:** Basic personal information.
- **emergencyContacts:** A list of saved contacts who receive SOS alerts.

Methods:

- **login():** Authenticates the user.
- **sendSOS():** Manually triggers an SOS alert.
- **viewReport():** Allows the user to view past emergency reports.

2. Responder Class

Purpose: Represents emergency contacts or responders who assist the user.

Attributes:

- **responderId, name, contact:** Identification and contact information.

Methods:

- `receiveAlert()`: Receives alerts with user details and location.
- `trackLocation()`: Tracks the real-time location of the user in emergency.

3. Admin Class

Purpose: Manages backend operations such as verifying responders and managing user data.

Attributes:

- `adminId, name`: Unique identification of the admin.

Methods:

- `verifyResponder()`: Confirms the legitimacy of a responder.
- `manageUsers()`: Oversees user and responder accounts.
- `viewAllReports()`: Accesses all system-wide incident reports.

4. SOSAlert Class

Purpose: Manages the process of creating and sending emergency alerts.

Attributes:

- `alertId, timeStamp, location, status`: Stores alert metadata.

Methods:

- `triggerAlert()`: Initiates the alert when the user presses SOS or falls.
- `updateStatus()`: Changes the status of the alert (e.g., pending, resolved).

Relations:

- Triggered by the **User**
- Uses **LocationService** to fetch GPS coordinates.

- Notifies the **Responder**
- Can also be triggered by the **FallDetector**
- Connected to **IncidentReport** for data logging

5. LocationService Class

Purpose: Provides real-time user location.

Methods:

- `fetchLocation()`: Returns the current GPS coordinates of the user.

6. Fall Detector Class

Purpose: Detects sudden movements or falls using sensor data.

Methods:

- `monitorMovement()`: Continuously tracks movement.
- `detectFall()`: Returns true if a fall is detected.

Relation: Automatically triggers the **SOSAlert** class when a fall is confirmed.

7. AudioRecorder Class

Purpose: Records audio during emergencies.

Methods:

- `startRecording()`: Begins capturing sound.
- `stopRecording()`: Ends the recording.
- `saveAudio()`: Saves the file locally or uploads to Firebase.

Relation: Activated by the **SOSAlert** class when an alert is triggered.

8. IncidentReport Class

Purpose: Stores the full record of an emergency incident.

Attributes:

- reportId, alertId, userId, audioFile: Stores all relevant info.

Methods:

- generateReport(): Compiles incident details.
- getDetails(): Returns data to user, responder, or admin.

Relations:

- Connected to **User, Responder, Admin**
- Created by the **SOSAlert** class after an event occurs

4.5.5 Activity diagram

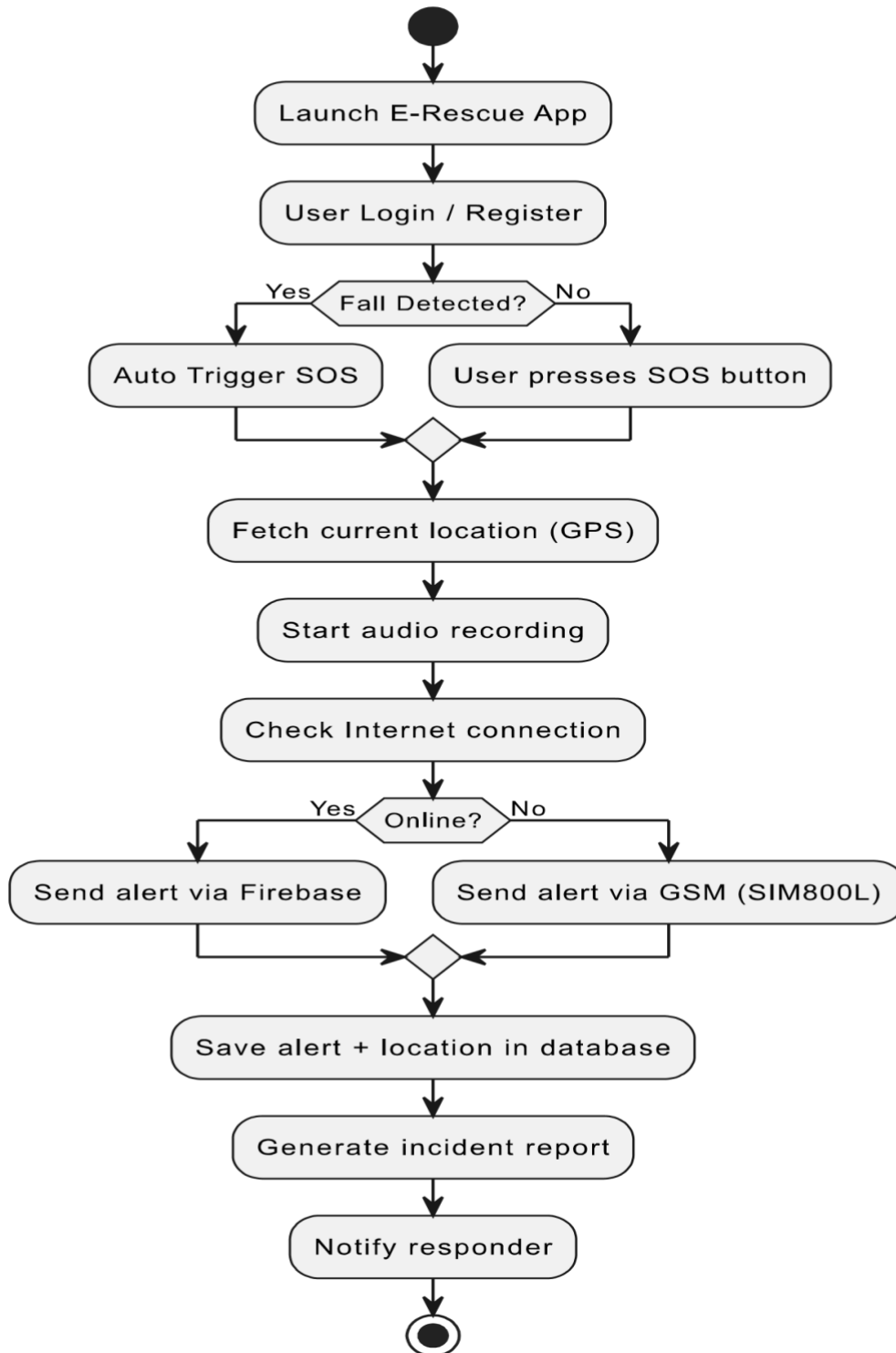


Figure 4.6: Activity Diagram

Description of Activity Diagram

The **activity diagram** of the **E-Rescue Application** represents the dynamic workflow of how the system responds during an emergency situation. It illustrates the **step-by-step flow of control** from the moment the app is launched to the point where an alert is sent and the incident is logged.

This diagram covers both **user-initiated actions** and **automatic system responses**, including decision-making based on **internet connectivity**. It clearly highlights how the app ensures an emergency alert is delivered whether the user has access to the internet or not.

Flow of Activities

1. Start / Launch App

- The process begins when the user opens the E-Rescue mobile application.

2. Login or Registration

- The user logs in to an existing account or registers as a new user. This step ensures user authentication and access to personal safety settings.

3. SOS Trigger Check (Decision Point)

- The system checks whether an emergency was automatically detected via the **fall detection sensor (MPU6050)**.
- If a fall is detected, the app automatically initiates an SOS alert.
- If no fall is detected, the user can manually press the SOS button to trigger the alert.

4. Fetch Location

- The app fetches the user's **current GPS location** using the built-in location service or ESP8266 (if hardware is used).

5. Start Audio Recording

- Simultaneously, the app begins recording audio to capture the surrounding environment for evidence, which can later be used in the incident report.

6. **Internet Connectivity Check (Decision Point)**

- The system checks whether the user is currently connected to the internet.

7. **Send Alert (Two Paths):**

If online:

- The app sends the SOS alert, including the location and audio, to Firebase (cloud services).

If offline:

- The alert is sent via GSM Module (SIM800L) in the form of an SMS to predefined emergency contacts.

8. **Save Alert in Database**

- The system stores alert details such as timestamp, location, and user ID in the Firebase real-time database.

9. **Generate Incident Report**

- A structured report is automatically created, which includes all emergency details, including fall detection status and recorded audio.

10. **Notify Responder**

- Finally, the designated responder (family member, medical responder, etc.) is notified through either Firebase (push notification) or SMS.

11. **End of Process**

- The emergency flow ends after the system has successfully handled the alert and sent all necessary data.

Chapter 5: Project Implementation

5.1. Programming language

Dart (with Flutter): Dart is an open-source, object-oriented programming language developed by Google. It is optimized for building fast, cross-platform mobile applications and is primarily used with the Flutter framework to develop apps for both Android and iOS from a single codebase. Dart is known for its clean syntax, strong typing, and efficient performance, making it ideal for developing real-time, responsive applications like the E-Rescue App.

5.2. Framework

5.2.1 Flutter is an open-source UI development framework created by Google. It enables developers to build high-performance, cross-platform mobile applications using a single Dart codebase. Flutter is well-suited for the E-Rescue App because it supports fast development, expressive user interfaces, and real-time responsiveness, which are essential for emergency-related features like SOS alerts and live location sharing.

5.2.2 Android Studio is the official integrated development environment (IDE) for Android app development. It provides a powerful set of tools for writing, testing, and debugging code efficiently. For the E-Rescue App, Android Studio supports Flutter integration and offers advanced emulators, performance monitoring, and real-device testing capabilities to ensure a smooth and stable application.

5.2.3 Firebase is a cloud-based platform by Google that offers backend services such as real-time databases and push notifications. In the E-Rescue App, Firebase plays a key role in handling emergency data transmission, sending real-time alerts, and storing audio or incident reports securely.

5.3. System Requirement

5.3.1. Android Application Development:

To ensure a smooth development experience for the Android application the following system specifications are recommended:

Operating System: A modern operating system such as Windows 10 or 11, macOS, or a stable Linux distribution is required for compatibility with development tools.

Processor: Intel Core i5 (10th generation or higher) with at least 12 logical processors is recommended to efficiently handle Flutter builds, emulator instances, and backend server tasks.

RAM: 16 GB of RAM is recommended for smooth multitasking across Android Studio, emulators, VS Code, and browser tools. This helps ensure faster builds and responsive performance.

Disk Space: A SolidState Drive (SSD) with at least 4 GB of free space is required to install Android Studio, emulators, dependencies, and project files. More space is recommended for backups and versioning.

Software: Android Studio is the recommended IDE for building and testing the mobile application. However, the backend server, can also be developed and run using Visual Studio Code (VS Code), which offers lightweight and flexible development support for server-side logic.

5.4. Chapter Summary

This chapter explains the technical foundation of the E rescue project. Dart was used for frontend and backend development through Flutter, which enabled cross-platform mobile app creation. On the backend it handles server-side logic, and Firebase was integrated for real-time database and secure storage. The recommended development environment includes a modern OS, Intel i5 (10th Gen or higher) processor, 16 GB RAM, and SSD storage to ensure optimal performance

Chapter 6: Software Testing & Maintenance

6.1 Deriving Test

To ensure the reliability and effectiveness of the E-Rescue App, various test cases were derived from the functional and non-functional requirements. These test cases cover critical functionalities such as SOS alert triggering, real-time location sharing, fall detection, audio recording, and offline operation. The goal of deriving these tests is to validate that the app responds correctly in emergency scenarios and maintains consistent behavior across devices and conditions.

6.2 Test Environment

The E-Rescue App was tested on the following environment:

- **Devices:** Android smartphones (Android 10 and above)
- **Emulators:** Android Studio AVD (API 30+)
- **Operating System (Dev):** Windows 10 and Ubuntu 20.04 LTS
- **Development Tools:** Android Studio, Flutter SDK, Firebase Console
- **Network:** Wi-Fi and Mobile Data (3G/4G)

The environment simulates both real-world and edge-case conditions to ensure stable performance in emergency situations.

6.3 Testing identification

The primary components tested include:

- User Registration/Login
- Triggering the SOS alert
- Live location sharing
- Fall detection and auto alert
- Audio recording and playback

- Notification delivery via Firebase
- Offline functionality and local storage
- Responder panel data retrieval

Each module was tested for expected input/output behavior and exception handling.

6.4 Test Procedure

To ensure accurate validation of core functionalities, the following test procedure was applied during the testing phase of the E-Rescue App:

1. Installation & Launch Testing

- Install the E-Rescue APK on both real Android devices and emulators.
- Verify successful launch, splash screen display, and login screen visibility.

2. User Registration and Login Testing

- Create new test user accounts using dummy data.
- Log in with valid and invalid credentials to test error handling and Firebase Authentication.

3. Functional Feature Testing

- **SOS Button:** Tap the SOS button and verify if emergency alerts are sent to predefined contacts via Firebase Cloud Messaging.
- **Location Sharing:** Enable GPS and check if live location updates are displayed correctly on the Responder Panel.
- **Fall Detection:** Simulate fall events by dropping the device or shaking it rapidly to test auto-triggering of alerts.
- **Audio Recording:** Start a recording during an emergency; test if the recording is saved and accessible from Firebase Storage or local device memory.

4. Offline Mode Testing

- Disable internet connectivity and access offline features.
- Record audio, trigger SOS, and verify that data is stored locally and synced to Firebase once internet is restored.

5. UI/UX Testing

- Evaluate the app's user interface on various screen sizes and resolutions.
- Check visibility of critical buttons like SOS, tips, and responder panel.

6. Compatibility & Device Testing

- Run the app on different Android versions (e.g., Android 10, 11, 12) and device models.
- Check performance and responsiveness across devices with different RAM/processor capabilities.

7. Responder Panel Testing

- Use a test responder account to access real-time alerts, view location data, and download incident reports.

6.5 Testing technique

To ensure the E-Rescue App works properly in real-life emergency situations, several practical testing methods were used:

1. Feature Testing:

Each main feature of the app such as the SOS button, live location sharing, audio recording, and fall detection was tested to make sure it performs its job correctly when used by the user.

2. Step-by-Step Testing:

We followed each user action step by step, such as registering, logging in, sending alerts, and recording audio, to check that every part of the process works smoothly without errors.

3. Real Device Testing:

The app was tested on multiple Android smartphones with different screen sizes and versions to make sure it looks good and runs well on all supported devices.

4. Offline and Low Battery Testing:

The app was used in offline mode and with low battery to confirm that critical functions like SOS alerts and fall detection continue to work when the device is not fully charged or connected to the internet.

5. Emergency Scenario Testing:

Simulated emergency situations (like sudden movement or fall) were performed to verify that the app responds immediately by sending alerts and activating safety features.

6. User Experience Testing:

The app was tested for ease of use, making sure that all buttons, screens, and actions are simple, clear, and easy to understand especially in stressful situations.

6.6 Test Cases

Test Case ID	Feature	Description	Expected Result	Status
TC01	Login	Test login with valid credentials	User logged in successfully	Pass
TC02	Login	Test login with invalid credentials	Error message displayed	Pass

TC03	Registration	Register with valid information	Account created successfully	Pass
TC04	Registration	Register with missing required fields	Form validation error shown	Pass
TC05	SOS Button	Tap SOS button while online	SOS sent via Firebase and alert generated	Pass
TC06	SOS Button	Tap SOS button while offline	SOS sent via GSM (SIM800L)	Pass
TC07	Location Sharing	Test location tracking after SOS	Location sent to database/responder	Pass
TC08	Location Sharing	Location module off during SOS	Location fetch fails, appropriate error shown	Pass
TC09	Audio Recording	Start recording during SOS	Audio file recorded and stored	Pass
TC10	Audio Recording	Microphone permission denied	App shows permission request message	Pass
TC11	Fall Detection	Simulate fall detection	Automatic SOS triggered	Pass
TC12	Fall Detection	Normal movement	No SOS is triggered	Pass
TC13	Emergency Contact Notification	Alert is sent to emergency contact	SMS or Firebase message delivered	Pass
TC14	Incident Report	Generate incident report after SOS	Report is created with time, location, status	Pass
TC15	Weather Alerts	Enable weather alert notifications	Alerts are received based on weather service	Pass
TC16	Weather Alerts	Disable weather alerts	No weather notifications received	Pass

TC17	Danger Zone Notification	User enters geofenced area	Notification triggered	Pass
TC18	Danger Zone Notification	User in safe zone	No notification shown	Pass
TC19	Offline Mode	Use app without internet	Basic features like SOS & SMS work	Pass
TC20	Offline Mode	Submit report offline	Report queued or stored locally	Pass
TC21	Family Circle	Add member to family circle	Member added and receives notifications	Pass
TC22	Family Circle	Remove member from family circle	Member removed and no longer receives alerts	Pass
TC23	Personal Safety Tips	View safety tips	Tips are displayed correctly	Pass
TC24	Emergency Guides	Open emergency guide	Guide opens with correct content	Pass
TC25	Login Session Timeout	Stay idle after login	Auto logout or session timeout message	Pass
TC26	GSM Module	Test GSM sends message when network is down	Message delivered via SMS	Pass
TC27	Cloud Storage	Upload audio file	Audio stored on Firebase Cloud	Pass
TC28	Authentication	Unauthorized user tries to access app	Access denied	Pass
TC29	User Logout	Test logout functionality	User is logged out and redirected to login screen	Pass
TC30	App Crash Recovery	Restart app after crash during SOS	App resumes with saved state or prompts user to retry	Pass

6.7 Chapter Summary

This chapter focused on the testing and maintenance processes of the E-Rescue App. Various tests were designed and executed to ensure that all core features such as SOS alerts, real-time location sharing, fall detection, and offline functionality perform reliably in emergency situations. The testing was conducted in different environments, including both real devices and emulators, to validate the app's stability and compatibility. Simple yet effective testing techniques were used to assess usability, responsiveness, and accuracy. All major test cases passed successfully, confirming that the application is ready for deployment and real-world use. Regular maintenance and future testing will help keep the app secure, up-to-date, and reliable for its users.

Chapter 7: Conclusion and Future Work

7.1. Discussion

The E-Rescue App was developed to provide a fast, reliable, and user-friendly emergency response solution. By integrating features such as an SOS alert system, real-time location sharing, fall detection, audio recording, weather alerts, and a dedicated responder panel, the app aims to increase personal safety and assist users during critical situations. Throughout the development process, modern technologies like Flutter, Firebase, and device sensors were utilized to ensure efficiency and real-time responsiveness. Rigorous testing confirmed that the app performs well in various scenarios, including offline conditions and low battery states.

7.2. Conclusion

The project successfully achieved its goal of creating an emergency response mobile application that empowers users during emergencies. The E-Rescue App allows individuals to quickly notify their trusted contacts, share their live location, and document incidents, all within a single, intuitive interface. With core features functioning smoothly and real-time connectivity established through Firebase, the app demonstrates the potential of mobile technology in improving public safety. Overall, the app meets its functional and non-functional requirements and is ready for real-world deployment with room for future enhancements.

7.3. Limitations

Despite its strengths, the E-Rescue App has a few limitations:

- I. It currently supports only Android devices.
- II. Features like fall detection and geofencing depend heavily on sensor accuracy, which may vary by device.
- III. Without internet connectivity, real-time features like alerts and location sharing are delayed until reconnection.

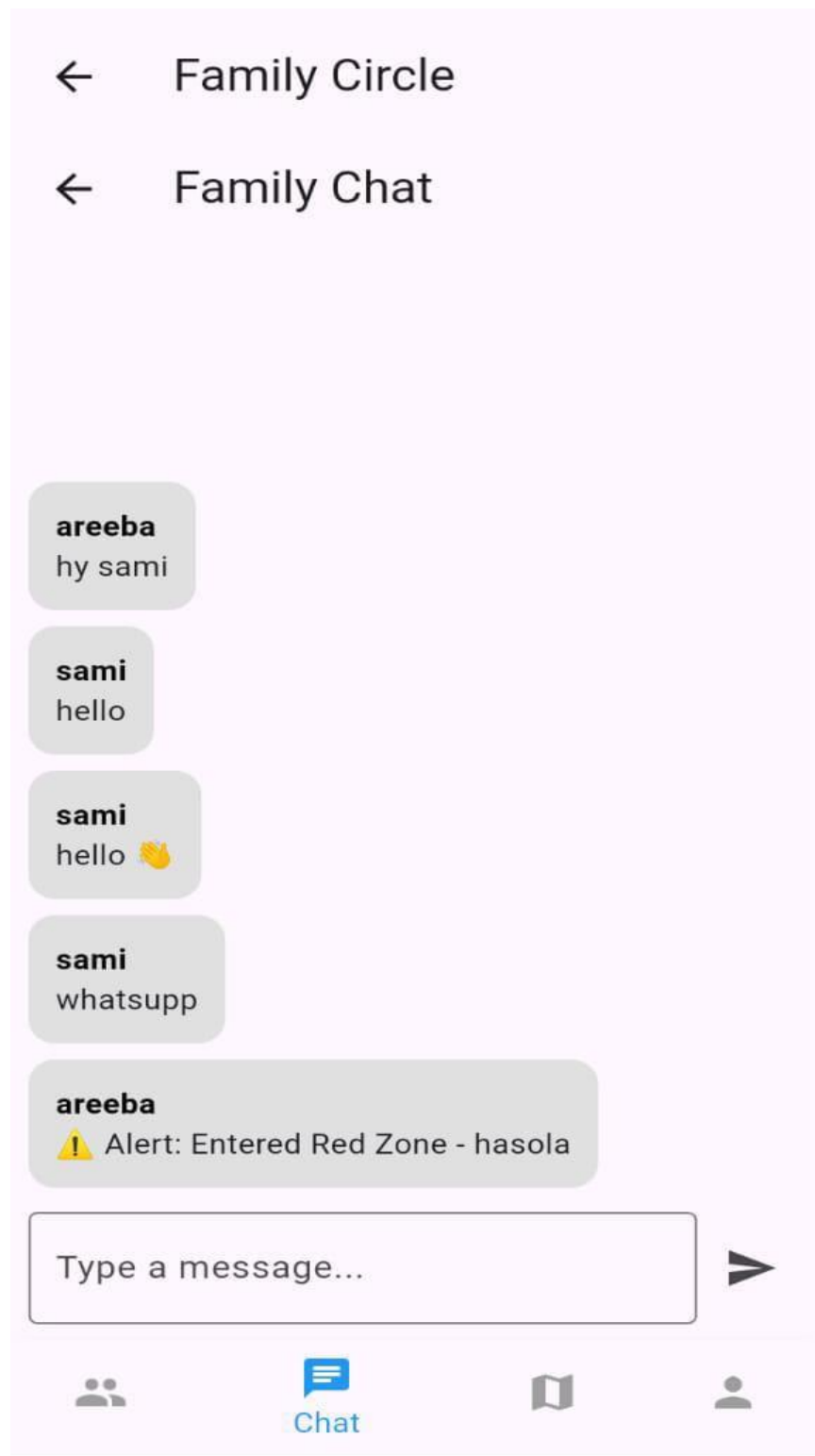
- IV. The app lacks multi-language support, which may limit accessibility for non-English-speaking users.

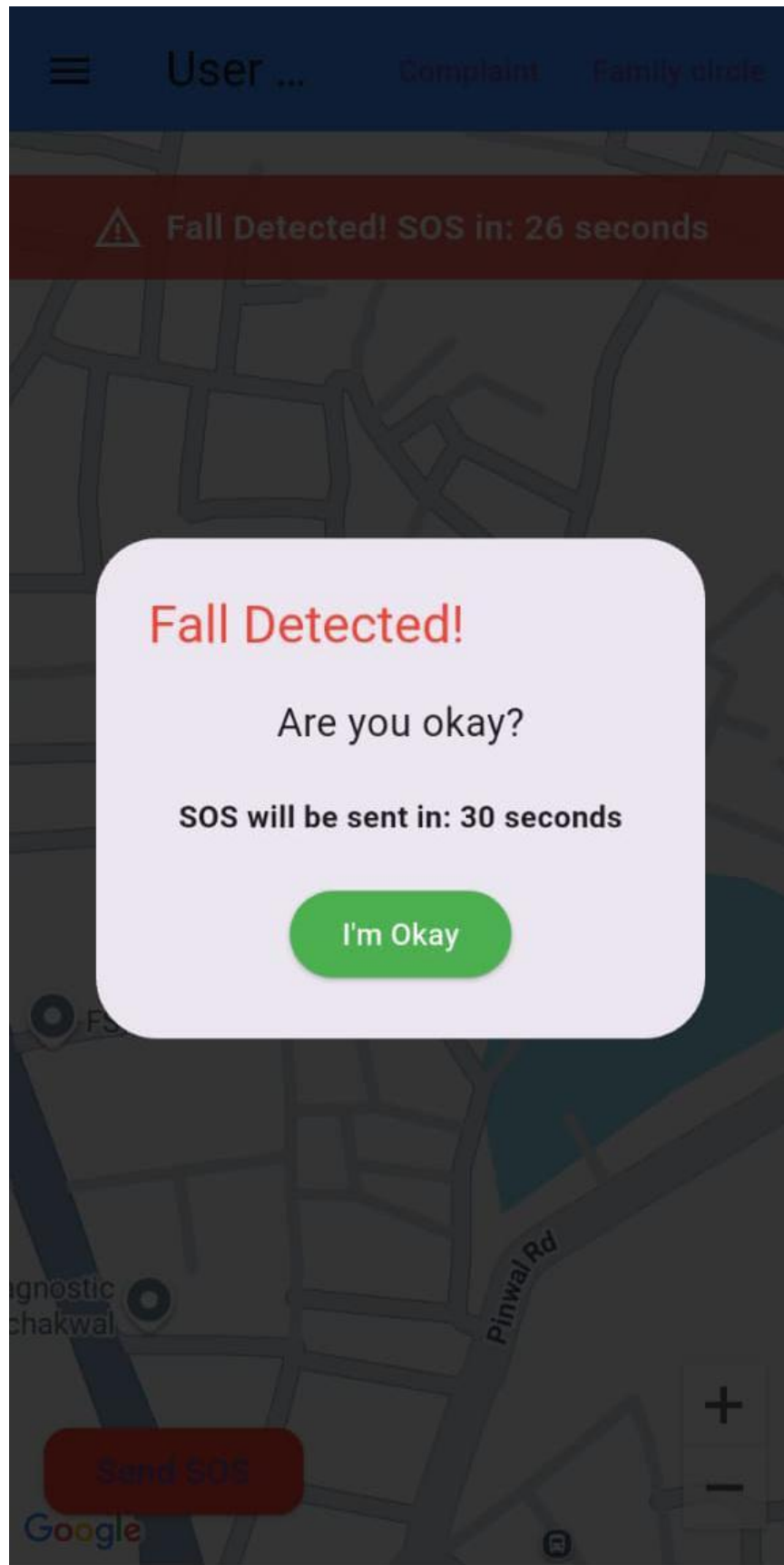
7.4. Future work

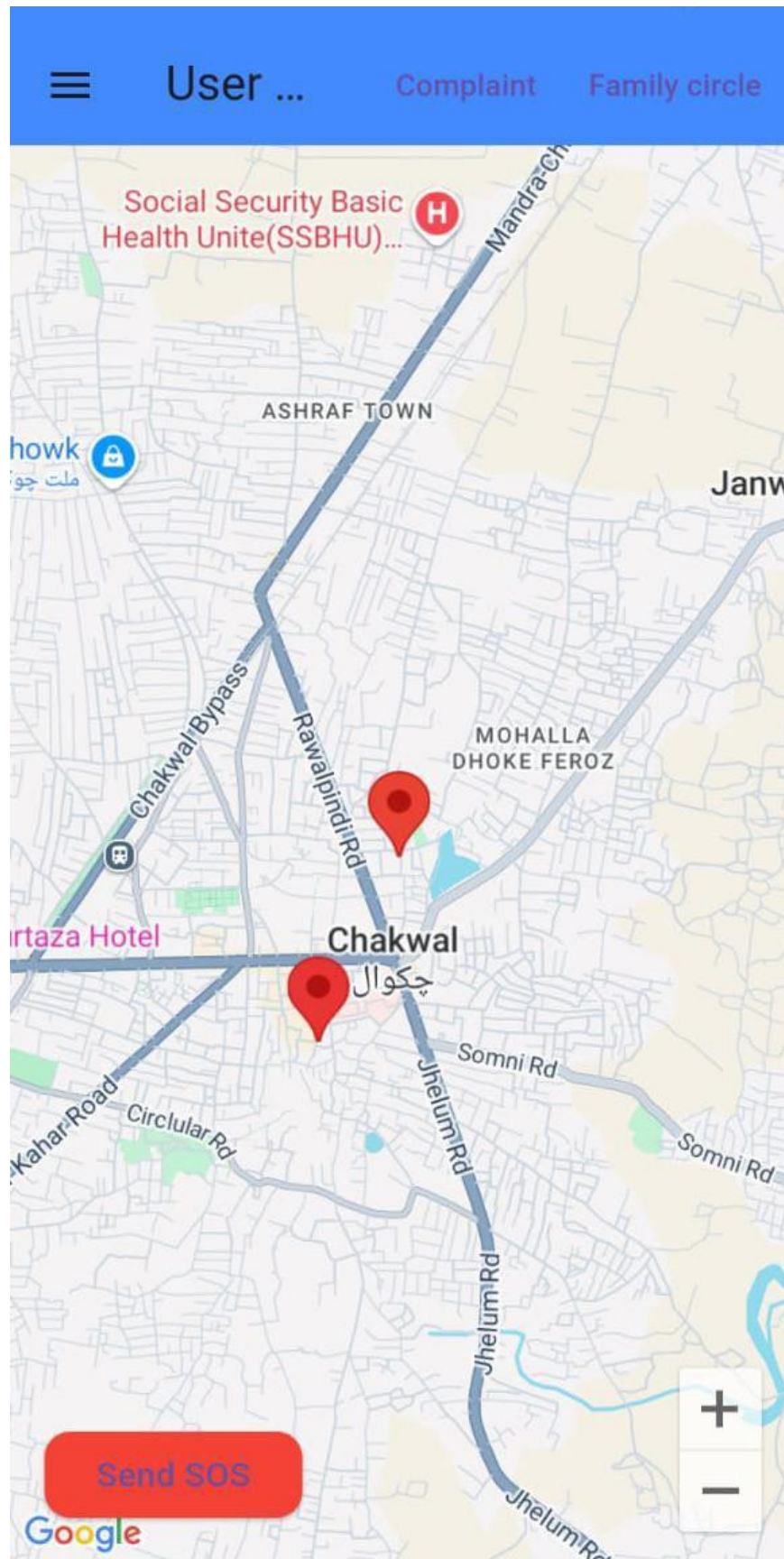
Several improvements and new features are planned for future versions of the app:

- **iOS support** to expand the user base across more devices.
- **AI-powered detection** for recognizing emergencies automatically through motion and sound patterns.
- **Two-way communication** between users and responders for better coordination during emergencies.
- **Multi-language support** to make the app accessible to users in different regions.
- **Wearable integration** (e.g., smartwatches) for more accurate fall detection and quick SOS activation.
- **Advanced analytics and reporting** to help emergency services better understand trends and improve response strategies.

App Screens

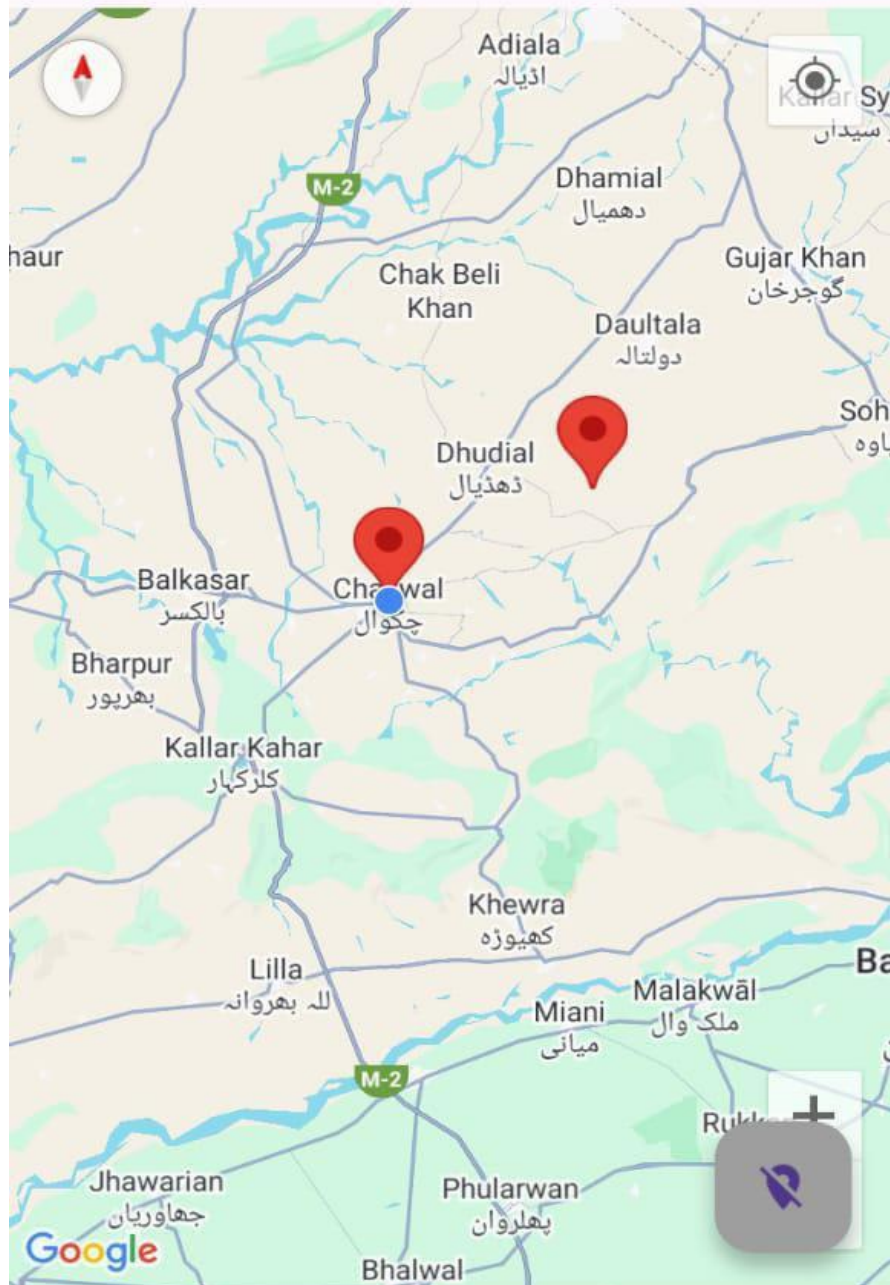






← Family Circle

← Family Locations



Welcome, sami

Email: user123@gmail.com

Role: User



Logout



Services



Weather



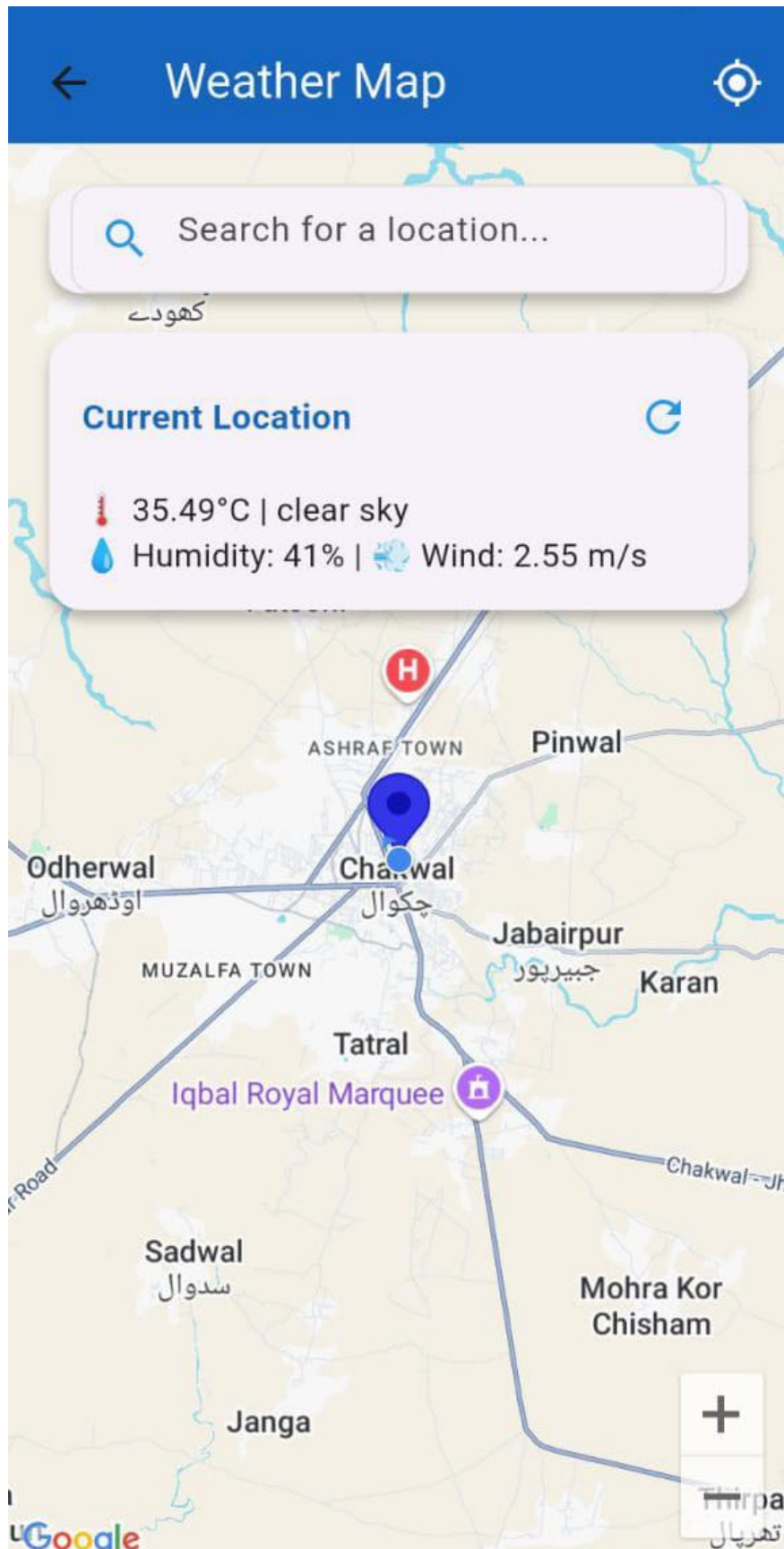
Battery



Request



First_Aid





First Aid Topics



CPR



Bleeding



Burns



Choking



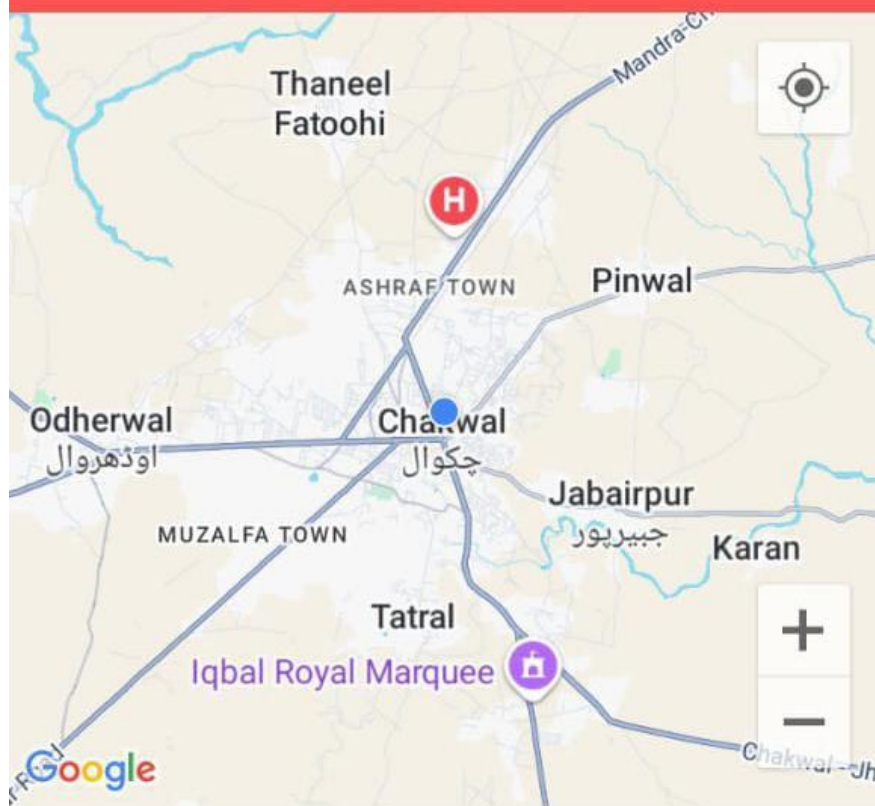
Poisons



Bites



Nearby Mechanics



Areeba Fatima



Expertise:

Status: offline

ETA: Not Available

sami



Expertise: General Mechanic

Status: online

ETA: 0 min

sami



Expertise:

Status: offline

ETA: Not Available

b



Community Forum

Share Your Complaints, Reviews, or Opinions

Write your complaint, review, or opinion here...

Submit Post

zaroonkhan123@gmail.com

I lost my phone in uni

Posted at: 2025-07-11 11:58:54.396

ali123@gmail.com

hi

Posted at: 2025-04-07 15:34:28.252

laylomakhan987@gmail.com

I need your help !!

Posted at: 2025-02-18 01:16:16.255

☰

Admin Home

Complaint

Refresh Accidents

 **Accident at Chakwal,
Pakistan** →
2025-07-13T12:29:12.975879

 Accident at Chakwal,
Pakistan →
2025-07-13T12:29:12.973347

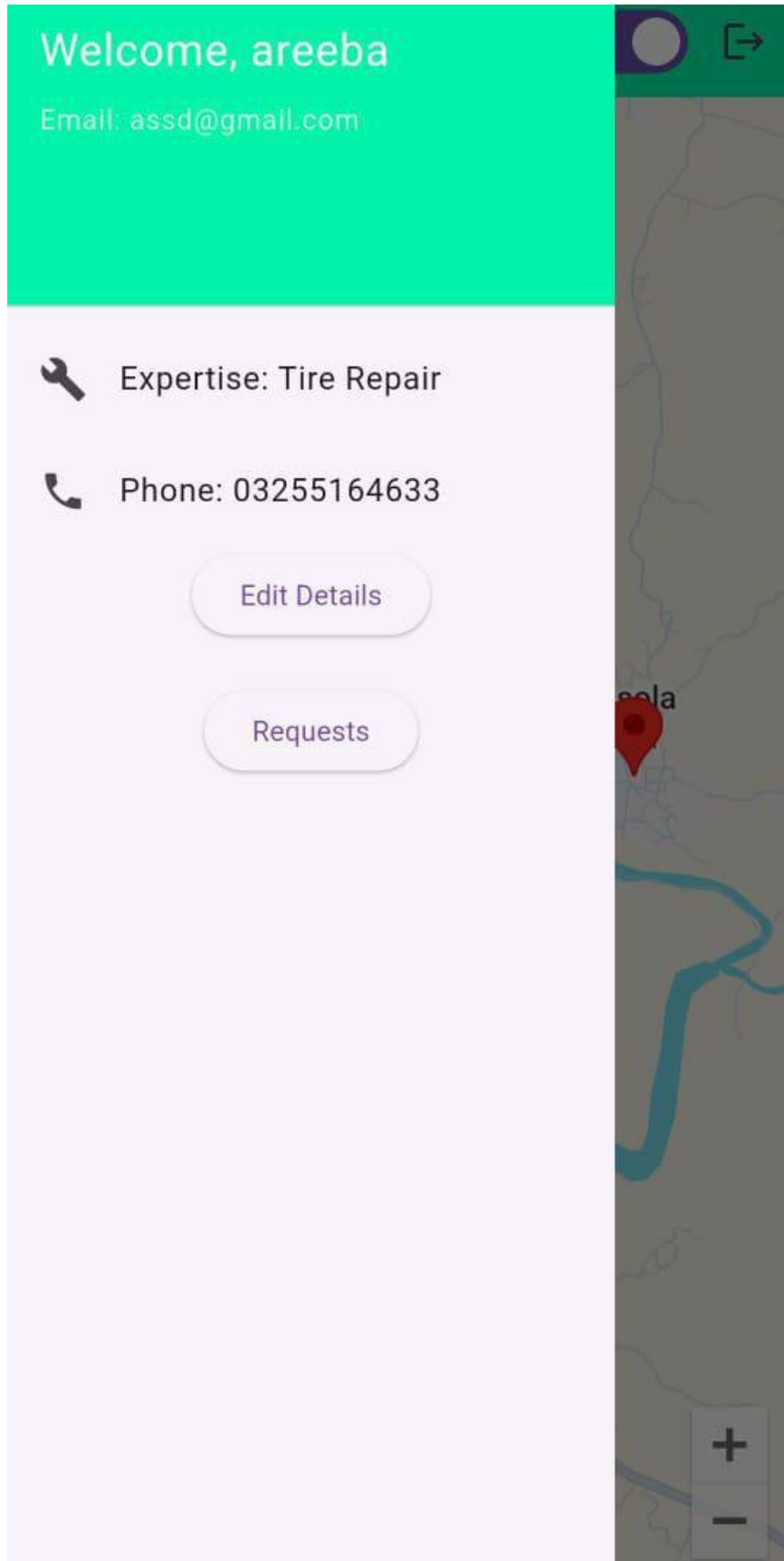
 Accident at Chakwal,
Pakistan →
2025-07-13T12:27:51.422637

 Accident at Chakwal,
Pakistan →
2025-07-13T12:27:51.419900

 Accident at Hasola, Pakistan →
2025-07-13T11:00:05.623213

 Accident at Chakwal,
Pakistan →
2025-07-12T18:51:50.288891

 Accident at Chakwal,





Service Requests

From User ID: TvZUKPuUaAQsr1ZD93otQDBWKK52

Status: accepted

Date: 2025-07-13

ACCEPTED

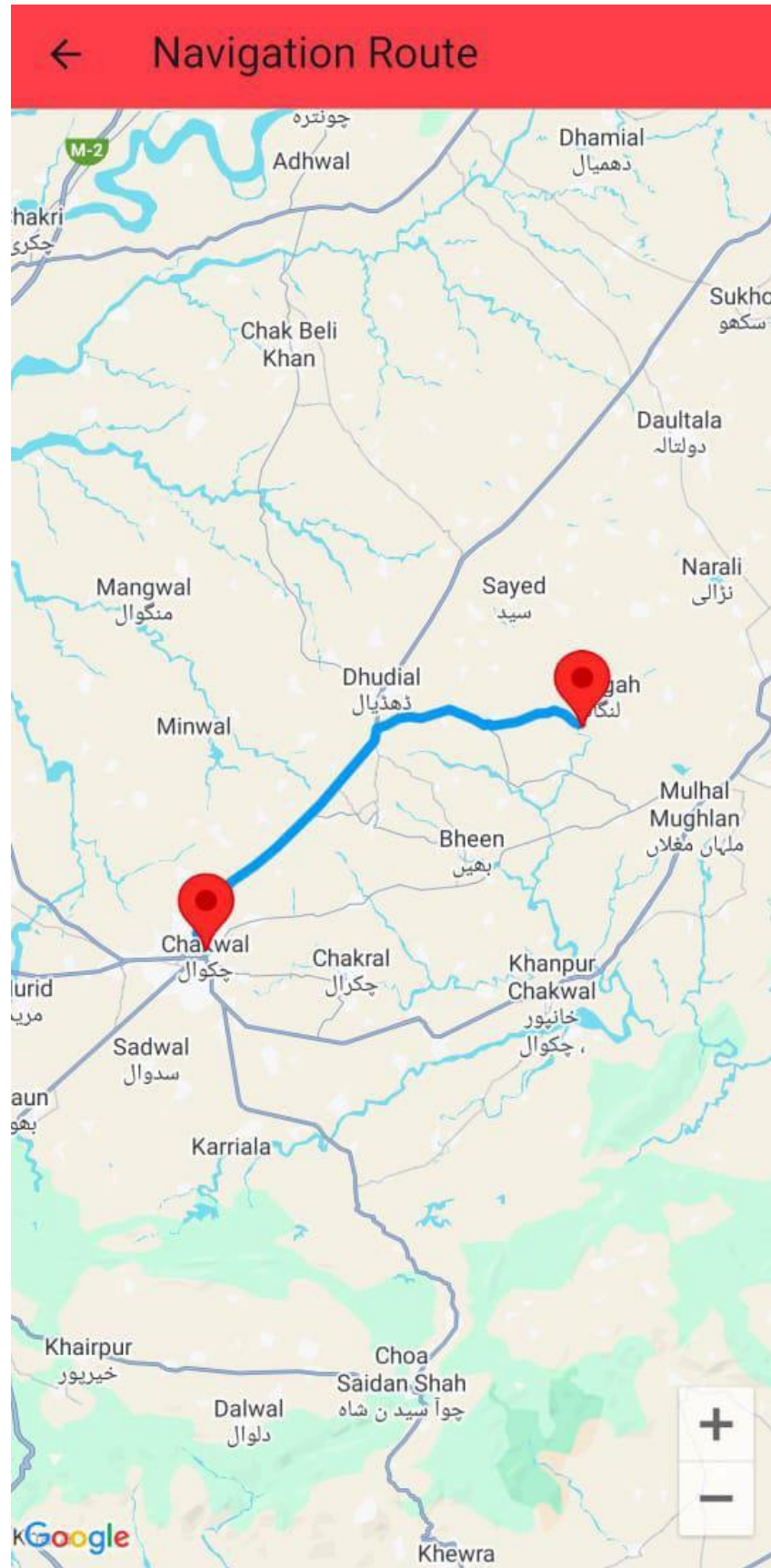
From User ID: TvZUKPuUaAQsr1ZD93otQDBWKK52

Status: pending

Date: 2025-07-13

Accept

Reject





References

- [1] <https://play.google.com/store/apps/details?id=com.life360.android.safetymapd>
- [2] <https://play.google.com/store/apps/details?id=com.ifirstaid>
- [3] <https://play.google.com/store/apps/details?id=uk.ucsoftware.panicbuttonpro>
- [4] <https://play.google.com/store/apps/details?id=com.hoboetech.rescue.consumer>
- [5] <https://play.google.com/store/apps/details?id=com.bipper.app.bsaf>
- [6] <https://ieeexplore.ieee.org/abstract/document/10187320>
- [7] <https://www.sciencedirect.com/science/article/pii/S2542660524002142>